> 本笔记基于 MIT OCW 6.006《Introduction to Algorithms》(Spring 2020) 的公开讲义、习题与考试材料整理，并对照 CLRS《Introduction to Algorithms》的章节结构。所有复杂度、接口、结论均可在文末「参考来源」列出的 PDF 中逐条核对；凡属我个人的工程经验、推断或课外补充，均在文中显式标注「推断」「工程补充」或「课外」。访问日期统一为 2026-11-02。

0.1 这门课的定位
0.2 先修要求（OCW 大纲原文口径）
0.3 评分构成（OCW 大纲原文数字）
0.4 考试结构（从真题 PDF 封面与目录表读出）
0.5 讲次地图（按 OCW calendar 页的周次）
0.6 与 CS170 笔记的分工
1.1 问题（Problem）
1.2 算法与正确性
1.3 效率与 Word-RAM 模型
1.4 数据结构 = 存储 + 接口
1.5 生日匹配：从 $O(n^2)$ 到 $O(n)$
1.6 复杂度与实现陷阱
2.1 问题：动态序列怎么做到「两端快、中间慢」
2.2 接口（讲义原文表格）
2.3 三种实现与代价
动态数组的核心技巧
删除与抖动
2.4 实现：带扩容/缩容的动态数组
2.5 实现陷阱
3.1 问题
3.2 接口视角
3.3 三种 $O(n^2)$ / $O(n \log n)$ 排序
选择排序（Selection Sort）
插入排序（Insertion Sort）
归并排序（Merge Sort）
3.4 复杂度汇总
3.5 实现陷阱
4.1 问题：find 能不能比 $O(\log n)$ 更快
4.2 直接寻址数组（Direct Access Array）
4.3 哈希表与冲突
4.4 全域哈希（Universal Hashing）
4.5 复杂度表（讲义原文）
4.6 实现骨架
4.7 实现陷阱
5.1 问题：能不能突破 $n \log n$
5.2 直接寻址排序（Direct Access Array Sort）
5.3 元组排序与稳定性
5.4 基数排序（Radix Sort）
5.5 实现陷阱
6.1 问题：把 $O(h)$ 变成 $O(\log n)$
6.2 结构：三个指针的结点
6.3 导航操作（都是 $O(h)$）
6.4 动态操作（只增删叶子）
6.5 两种应用：Set 与 Sequence
应用一：Set = BST
应用二：Sequence = 带 size 增广的树
6.6 复杂度
6.7 实现陷阱
7.1 问题：让 $h = O(\log n)$
7.2 旋转：改结构但不改遍历序
7.3 平衡条件与高度界
7.4 局部再平衡（4 种情形）
7.5 全局再平衡
7.6 增广（augmentation）的系统方法
7.7 插入/删除的实现骨架
7.8 复杂度与用途
7.9 实现陷阱
8.1 问题：只要最大/最小，能不能更简单
8.2 结构：数组即完全二叉树
8.3 两个核心操作
8.4 线性建堆（build heap in $O(n)$）
8.5 堆排序（原地）
8.6 实现陷阱
9.1 问题
9.2 表示：邻接表（adjacency list）

9.3 路径与距离
9.4 BFS：按距离分层
9.5 复杂度
9.6 实现陷阱
10.1 问题
10.2 算法
10.3 三个应用
连通分量（Connected Components）
拓扑排序（Topological Sort）
环检测（Cycle Detection）
10.4 复杂度
10.5 实现陷阱
11.1 问题
11.2 松弛（relaxation）框架
11.3 DAG 松弛
11.4 父指针的事后重建
11.5 复杂度对照
11.6 实现陷阱
12.1 问题
12.2 两个关键引理
12.3 分层图构造（讲义的讲法）
12.4 复杂度
12.5 实现陷阱
13.1 问题
13.2 关键观察
13.3 需要的接口：可变优先队列
13.4 正确性（一句话版）
13.5 复杂度（讲义原文表）
13.6 Python 实现：懒删除（lazy deletion）
13.7 实现陷阱
14.1 问题
14.2 重赋权（reweighting）
14.3 算法
14.4 复杂度
14.5 实现陷阱
15.1 问题
15.2 SRT BOT：六步框架（本课最重要的模板）
15.3 Fibonacci：从指数到线性
15.4 DAG 最短路就是 DP
15.5 Bowling：一维 DP 的样板
15.6 复杂度与陷阱
16.1 问题
16.2 最长公共子序列（LCS）
16.3 最长递增子序列（LIS）与「加约束」
16.4 交替取硬币（zero-sum 博弈）
16.5 复杂度与陷阱
17.1 把 Bellman–Ford 看成 DP（状态扩张 Θ(n) 倍）
17.2 Floyd–Warshall：常数分支的 APSP
17.3 算术括号化（子串 DP + min/max 双状态）
17.4 钢琴指法（状态 = 局部决策）
17.5 复杂度与陷阱
18.1 问题：什么叫「多项式」
18.2 切杆（Rod Cutting）
18.3 子集和（Subset Sum）
18.4 三个层次的复杂度
18.5 陷阱
19.1 判定问题
19.2 三个类
19.3 NP：用证书定义
19.4 归约与 NP 完全
19.5 算法设计的三个归约套路（讲义原文表）
19.6 陷阱
20.1 选型决策（工程版）
20.2 动态数组的三个数字
20.3 哈希表的三个不变量
20.4 AVL 的四个数字
20.5 堆的两个公式与一个界
20.6 并查集（Union-Find）：6.006 之外的必备结构
21.1 总表
21.2 四条决策规则
21.3 比较模型的下界（一句话）
21.4 实现检查清单
22.1 表示与代价
22.2 遍历家族
22.3 最短路选型（本课最重要的一张表）
22.4 三个「坑王」
23.1 子问题的五种形状（讲义 L18 结尾的总结表）
23.2 子问题扩张/约束的四种力度（讲义原文分类）

23.3 分支因子与复杂度
23.4 复原方案（parent pointers）
23.5 DP 的五个常见错误
24.1 两份笔记的知识互补图
24.2 推荐学习顺序
24.3 三个「只在 6.006 出现」的知识点
24.4 两个「只在 CS170 出现」的知识点
25.1 复习清单（按讲次打勾）
25.2 自测题（16 题，答案在后）
25.3 答案
26.1 页面（访问日期均为 2026-11-02）
26.2 讲义 PDF
26.3 习题与考试 PDF（题号、分值均取自 PDF 正文）
26.4 抓取说明

第 0 章 课程概览与讲次地图

0.1 这门课的定位

6.006 是 MIT 本科的算法入门课（undergraduate，6 系）。用一句话概括它的教学目标，讲义 L01 开篇写得非常直白：**教你解决计算问题，并且能向别人证明你的解是正确且高效的**。注意这里有两个交付物——正确性（correctness）与效率（efficiency）——而 6.006 与同目录 `UCB_CS170_算法设计_学习笔记.md` 最大的差别在于：6.006 把「效率」落到**具体数据结构的实现**上，CS170 则把「正确性」落到**证明技巧**上。

课程的基本事实（全部来自 OCW 页面与讲义 PDF 的页眉/页脚）：

项目	内容
课程编号与名称	6.006 Introduction to Algorithms
学期	Spring 2020
授课教师	Prof. Erik Demaine、Dr. Jason Ku、Prof. Justin Solomon
材料形态	20 份讲义 PDF（L01–L20）+ 19 份复习课（recitation）讲义 + 9 份 problem set + 3 次 quiz + final
教材	CLRS（OCW 大纲列的是 **Third Edition**，ISBN 9780262033848，并注明 "not required"）
考核	Quiz 1 / Quiz 2 / Quiz 3 / Final / Problem Sets / Recitation

> 关于教材版本：OCW 教学大纲写的是 CLRS 第三版。你手上如果有第四版，绝大部分主题章节号是一致的，但**图论部分整体前移**：第四版把 Elementary Graph Algorithms 放在第 20 章、MST 第 21 章、单源最短路（SSSP）第 22 章、全源最短路（APSP）第 23 章，而第三版对应第 22、23、24、25 章。本笔记在涉及图论章节号时会同时给两版编号，其余主题只给一个编号（两版一致）。

0.2 先修要求（OCW 大纲原文口径）

先修课	大纲要求的能力
6.0001 Introduction to Computer Science and Programming in Python	用 Python 3 编程的基本经验
6.042J Mathematics for Computer Science	集合论、关系与逻辑、组合计数、证明、递归、数论、图论、概率

大纲还写了一条很硬的规则：所有学生都必须提交 **Problem Set 0**（先修自测），这份作业会被打字母等级，拿到 C 或以下需要与助教面谈后才能继续选课；它的分数不计入最终成绩，但不交就不能选课。PS0 的题目内容（集合运算、期望、模运算、两条归纳证明）正好就是 6.042J 的最小可用子集——如果你集合与归纳生疏，先补这一块，否则 L01 的正确性证明会直接卡住。

0.3 评分构成（OCW 大纲原文数字）

项目	占比
Quiz 1	20%
Quiz 2	15%
Quiz 3	10%
Final Exam	35%
Problem Sets	18%
Recitation	2%

合计 100%。可以看出两个信号：**期末考试占 35%**，是最大单项；**三次 quiz 合计 45%**，且权重递减（20/15/10），说明课程希望你把前期数据结构与图论的基础打牢，越往后越偏向「会用 DP 建模」而不是「背结论」。

0.4 考试结构（从真题 PDF 封面与目录表读出）

考试	时长	满分	允许带的资料	题目构成（题名 / 分值）
Quiz 1	120 分钟	120	1 张双面手写 cheat sheet	Information 2、Frequentest 8、Haphazard Heaps 10、Transforming Trees 10、Sorting Sock 20、Triple Sum 15、Where am i? 15、Methane Menace 20、Vapor Invite 20

考试	时长	满分	允许带的资料	题目构成（题名 / 分值）
Quiz 2	90 分钟	90	2 张双面	Information 2、Exact Edges 16、Color Cost 18、Orkscapade 18、Count Cycles 18、Bellham's Fjord 18
Quiz 3	60 分钟	60	3 张双面	Information 2、Pseudo-Circling 3、Number Scrabble 17、Limited-Unlimited 19、Office Hour Optimization 19
Final	180 分钟	180	3 张双面	Information 2、Decision Problems 40（10 小问）、Sorting Sorts 24、Pythagorean Quad 14、Animal Counting 20、Limited Connections 14、On the Street 14、RGB Graph 18、Separated Subsets 16、A Feast for Crowns 18

三条从真题里读出来的应试规律（可验证）：

1. **Quiz 1 全是数据结构与排序**（堆、树、字符串排序），单题分值 8–20，题量 9 道，时间 120 分钟——平均每题 13 分钟，最后两道 20 分的题留 45 分钟比较稳。
2. **Quiz 2 全是图**（含一道「恰好 k 条边的环」= Bellman-Ford 分层 DP 的变体），每题 16–18 分，题量只有 5 道，90 分钟意味着每道题要写完整的算法描述 + 正确性 + 复杂度。
3. **Final 的第一题固定是 10 道判断并说明理由**（40 分，每题 4 分：2 分答案 + 2 分理由，全空仍给 1 分）。这说明「会判断命题真假并给一句理由」是独立考核项，复习时不能只练大题。

Quiz 3 与 Final 的中后段题目多数是 DP 建模题，题干会明确要求：定义子问题、写递推、论证无环、给基例、由子问题构造原问题解、分析时间。这六步就是 L15 讲的 **SRT BOT** 框架，本笔记第 15 章会完整展开。

0.5 讲次地图（按 OCW calendar 页的周次）

周	讲义	主题	关键交付
1–2	L01–L04	引言、数据结构、排序、哈希	PS0、PS1
3–4	L05–L07	线性排序、二叉树、AVL	PS2、PS3
5–7	L08–L12	二叉堆、BFS、DFS、DAG 松弛、Bellman-Ford	PS4、PS5、**Quiz 1**
8–10	L13–L17	Dijkstra、Johnson、DP 一/二/三	PS6、PS7、**Quiz 2**
11–12	L18–L20	伪多项式、复杂度、总复习	PS8、**Quiz 3**
13–14	L21 + Final	Next Steps（无讲义 PDF）；期末考试	**Final**

讲次与知识块的对应关系（讲义 L01 结尾给的「如何解算法题」框架，其实就是全课目录）：

- **搜索问题 → 数据结构**：Static Array(L01)、Linked List / Dynamic Array(L02)、Sorted Array(L03)、Direct-Access Array / Hash Table(L04)、Balanced Binary Tree(L06–L07)、Binary Heap(L08)。
- **排序算法**：Insertion / Selection / Merge(L03)、Counting / Radix(L05)、AVL Sort(L07)、Heap Sort(L08)。

- **最短路算法**：BFS(L09)、DFS + Topological Sort(L10)、DAG Relaxation(L11)、Bellman–Ford(L12)、Dijkstra(L13)、Johnson(L14)、Floyd–Warshall(L17)。
- **自己设计递归算法**：Brute Force、Decrease & Conquer、Divide & Conquer、Dynamic Programming(L15—L18)、Greedy/Incremental。

0.6 与 CS170 笔记的分工

两份笔记不是替代关系，而是「实现层 / 证明层」的互补。下面这张表是我建议的使用方式：

主题	6.006（本笔记）重点	CS170 笔记重点	建议
渐近分析	Word-RAM 模型、常数因子、摊还成本	形式化 O/Ω/Θ 证明、极限判别	先 CS170 第 1 章，再回来看 L01 的机器模型
分治	只用归并排序与递归树做例子	Karatsuba、Strassen、最近点对、Master 定理	**直接读 CS170**，6.006 不讲 Master 定理（只在 R03 复习课出现）
排序	6 种排序的接口、稳定性、原地性、Python 实现	比较模型下界、决策树	6.006 的更全（多了计数/基数排序的实现细节）
哈希	全域哈希族、链地址法、动态重建	基本不讲	只读 6.006
平衡树	AVL 的旋转、augmentation、实现	不讲	只读 6.006
图	邻接表实现、BFS/DFS 模板、Dijkstra 的可变优先队列	SCC、2-SAT、MST	各取所需：SCC/MST 在 6.006 不覆盖
最短路	五种算法的限制条件与实现代价（含 Johnson）	Dijkstra/BF/FW 的正确性证明	6.006 多了 Johnson 与「分层图」视角
DP	SRT BOT 六步、子问题扩张/约束、伪多项式	背包、TSP、树上独立集	两边都读，6.006 的建模训练更系统
复杂度	P/NP/EXP、归约方向、NP-hard 举例	Cook—Levin、完整归约链	证明细节读 CS170，结论与举例读 6.006

一句话记忆：**6.006 回答「这个东西怎么写、多快、坑在哪」，CS170 回答「为什么它对、为什么不能更快」。** 第 1–19 章按讲义逐讲精讲（问题 → 接口 → 算法 → 复杂度 → 陷阱），第 20–23 章是四个横向专题，第 24 章讲衔接，第 25 章是复习清单与自测题（附答案），第 26 章是来源清单。

第 1 章 引言：问题、算法、正确性与效率（L01）

1.1 问题（Problem）

讲义把「问题」定义为一个**从输入到正确输出的二元关系**，并且强调：不可能把所有正确输出都列出来（输入太多），所以实际做法是给一个**可验证的谓词（verifiable predicate）**，正确输出必须满足它。

6.006 只研究**输入规模可以任意大**的问题。反例（讲义原例）："这间教室里有没有两个同一天生日的学生？"——输入固定，属于小实例，不是本课对象。正例："给定任意 n 个学生，是否存在一对同生日？"——这才是。

1.2 算法与正确性

算法是**把每个输入映射到唯一输出的确定性过程**；若对每个输入都返回正确输出，则算法解决了该问题。

正确性怎么证？讲义给的路子是：程序长度固定，小输入可以穷举，但任意大的输入必须靠**循环或递归**，于是必须用**归纳法（induction）**。这就是「为什么递归是计算机科学的核心概念」的课程级回答。

生日匹配算法的归纳证明骨架（讲义原文结构）：

- 对 k 归纳：k 是已录入记录的学生数。
- 归纳假设：若前 k 个学生中存在同生日对，则在面试第 k+1 个学生之前就已经返回。
- 基例 k=0：前 0 个无同生日对，成立。
- 归纳步：若前 k_0 个已有匹配，由归纳假设已返回；否则前 k_0 个无匹配，那么前 k_0+1 个若有匹配，必包含第 k_0+1 个，算法正好检查了它与前 k_0 个的生日。

1.3 效率与 Word-RAM 模型

效率不能直接测时间（机器相关），所以**数固定时间操作的个数**，并让它依赖输入规模 n。讲义给的直觉表（1 GHz 单核、每周期一个操作）：

输入 n=1000	Θ(1)	Θ(log n)	Θ(n)	Θ(n log n)	Θ(n^2)	Θ(n^c)	2^Θ(n^c)
操作数	1	≈10	1000	≈10,000	1,000,000	1000^c	2^1000 ≈ 10^301
时间	1 ns	10 ns	1 µs	10 µs	1 ms	10^(3c−9) s	10^281 千年

计算模型叫 **Word-RAM**：

- 机器字（machine word）是 w 个比特；
- 内存是可寻址的字序列；
- 处理器对 O(1) 个字支持常数时间操作：整数算术（+ − × // %）、逻辑运算、位运算、按地址读写；
- 内存地址必须能覆盖整个内存，所以要求 $w \geq \log_2 n$（这也是为什么"字长"和"n"绑在一起）。32 位字 → 最多约 4 GB 内存；64 位字 → 最多约 16 EB。

> **实现陷阱 1–1**：Word-RAM 假设「一个机器字上的算术是 O(1)」，但 Python 的 `int` 是任意精度大整数。算 2^1000 或第 n 个 Fibonacci 数时，加法不是 O(1) 而是 O(⌈n/w⌉)（讲义 L15 明确提了这点：Fibonacci 总代价 O(n + n^2/w)）。做复杂度分析时要么假设数值落在字长内，要么显式乘上大整数代价。

1.4 数据结构 = 存储 + 接口

- 数据结构（data structure）：存储非常量数据并支持一组操作的方式。
- 接口（interface，也叫 API / ADT）：支持的操作集合，是**规格说明**（问题）；数据结构是**表示**（解）。
- 本课只有两大接口：**Sequence**（顺序是外加的/extrinsic）与 **Set**（顺序由 key 内蕴/intrinsic）。

Static Array 就是最小的 Sequence 实现：`StaticArray(n)` 用 Θ(n) 初始化；`get_at(i)` / `set_at(i,x)` 都是 Θ(1)。Python 的 tuple 类似它，Python 的 list 则是动态数组（第 2 章）。

1.5 生日匹配：从 O(n^2) 到 O(n)

讲义给的是教学版：外层遍历学生，内层线性扫描已有记录，总时间 O(n^2)：

```
def birthday_match(students):
    record = [None] * len(students)          # StaticArray, O(n)
    for k, (name1, b1) in enumerate(students):
        for i in range(k):                   # 内层 k 次 => 总计 O(n^2)
            if record[i][1] == b1:
                return (name1, record[i][0])
        record[k] = (name1, b1)
    return None
```

时间：O(n) + Σ_{k=0}^{n−1}(O(1) + k·O(1)) = O(n^2)。

换数据结构就换复杂度——把 `record` 换成哈希表（第 4 章）后，内层查找变成期望 O(1)，总时间降到期望 O(n)。这个改写是我加的（讲义本身停在 O(n^2) 并追问"能否更快"），它正好演示了全课的主线：*先选接口，再选实现，复杂度是实现的属性*。

1.6 复杂度与实现陷阱

操作/写法	讲义口径	Python 现实
数组按下标读写	Θ(1)	`list[i]` 是 Θ(1)，但取的是对象指针
分配 n 个元素的数组	Θ(n)	`[None]*n` 是 Θ(n)，且是堆分配
比较两个数	Θ(1)	int 比较在字长内 Θ(1)，超长整数不是

操作/写法	讲义口径	Python 现实
递归深度	未讨论	默认上限 1000 层，DFS 深图会 `RecursionError`

- **陷阱 1–2**：把「循环次数」当复杂度。内层循环上界是 k 不是 n，求和后才是 n^2；写成 O(n·n) 结论对但推导错，考试会扣过程分。
- **陷阱 1–3**：忘记 build 的成本。`StaticArray(n)` 初始化是 Θ(n)，很多「O(1) 空间」的说法实际是 O(n) 初始化。
- **陷阱 1–4**：Python 的 `list` 既是 Sequence 又是「可变长」，但 `insert(0, x)` 是 O(n)——它不是链表。

第 2 章 数据结构与序列接口（L02）

2.1 问题：动态序列怎么做到「两端快、中间慢」

需求是维护一个序列 ($x_0, x_1, \ldots, x_{n-1}$)，支持按下标读写、任意位置插入删除、以及首尾插入删除。难点在于：数组的按下标访问是 Θ(1)，但插入删除要搬移；链表的插入删除是 Θ(1)，但按下标访问要走路。

2.2 接口（讲义原文表格）

类别	操作	语义
Container	`build(X)`	用可迭代对象 X 建序列
Container	`len()`	返回元素个数
Static	`iter_seq()`	按序列顺序逐个返回
Static	`get_at(i)`	返回第 i 个元素
Static	`set_at(i, x)`	把第 i 个元素替换为 x
Dynamic	`insert_at(i, x)`	把 x 插成第 i 个
Dynamic	`delete_at(i)`	删除并返回第 i 个
Dynamic	`insert_first(x)` / `delete_first()`	首部插/删
Dynamic	`insert_last(x)` / `delete_last()`	尾部插/删

特例接口：**栈（stack）** = `insert_last` + `delete_last`；**队列（queue）** = `insert_last` + `delete_first`。

2.3 三种实现与代价

讲义的三张复杂度表（最坏情况，$O(\cdot)$）：

数据结构	build(X)	get_at / set_at	insert_first / delete_first	insert_last / delete_last	insert_at / delete_at
Array（定长、始终填满）	n	1	n	n	n
Linked List（单链、只有 head）	n	n	1	n	n
Dynamic Array	n	1	n	**1（摊还）**	n

动态数组的核心技巧

- 维护**填充率（fill ratio）** r = 元素数 / 容量，$0 \leqslant r \leqslant 1$。
- 数组满（$r = 1$）时，一次性分配 $\Theta(n)$ 额外空间，把 r 降到 r_i（讲义例子 1/2）。
- 于是要再插入 $\Theta(n)$ 个元素才会再次扩容，**任意 $\Theta(n)$ 次操作总共 $\Theta(n)$**，单次摊还 $\Theta(1)$。

删除与抖动

- 从尾部删除本身是 $\Theta(1)$，但空间会浪费，希望空间始终 $\Theta(n)$。
- 天真做法「太空就缩到 r=1」会导致插入/删除交替时反复搬运。
- 正确做法：当 $r < r_d$ 时缩到 r_i，且要求 **$r_d < r_i$**（讲义例子 $r_d = 1/4$、$r_i = 1/2$）。这样缩容后必须再删掉 $\Theta(n)$ 个元素才会再次触发。
- 讲义给的通用参数化：任意 $\varepsilon > 0$，可把额外空间限制在 $(1+\varepsilon)n$，取 $r_d = 1/(1+\varepsilon)$、$r_i = (r_d+1)/2$。

摊还分析的定义（讲义原文）：若任意 k 次操作的总代价 $\leqslant k \cdot T(n)$，则单次操作摊还代价为 $T(n)$。更精细的势能法在 6.046 讲。

2.4 实现：带扩容/缩容的动态数组

```
class DynamicArray:
    def __init__(self): self.A, self.n = [None], 0        # 容量 1 起
    def get_at(self, i): return self.A[i]                 # Θ(1)
    def set_at(self, i, x): self.A[i] = x                 # Θ(1)
    def _resize(self, cap):
        self.A = [self.A[i] if i < self.n else None for i in range(cap)]
    def insert_last(self, x):
        if self.n == len(self.A): self._resize(2 * len(self.A))   # 满则翻倍
        self.A[self.n] = x; self.n += 1
    def delete_last(self):
        x = self.A[self.n - 1]; self.A[self.n - 1] = None; self.n -= 1
        if len(self.A) > 1 and self.n / len(self.A) < 1/4:        # r_d = 1/4
            self._resize(max(1, int(self.n / (1/2)) + 1))         # 缩到 r_i = 1/2
        return x
```

`insert_at(i, x)` 与 `delete_at(i)` 的实现是「尾部腾挪 + 搬移」：先把 i 之后的所有元素右移一位（$\Theta(n-i)$），再写入。

2.5 实现陷阱

- **陷阱 2-1**：缩容阈值取 r_d = r_i 会抖动。必须 r_d < r_i，且中间留出足够「便宜操作」的缓冲。
- **陷阱 2-2**：`_resize` 里忘记把旧数组的元素逐个搬过去，或搬完忘记更新容量字段。
- **陷阱 2-3**：把「摊还 O(1)」当成「每次 O(1)」。实时系统里单次 $\Theta(n)$ 的搬运会造成长尾延迟，这是工程上真实存在的问题（推断：与 GC/内存分配器交互时更明显）。
- **陷阱 2-4**：迭代过程中修改容器。Python list 在遍历时 append 可能死循环，删除会跳元素；正确做法是先复制或倒序遍历。
- **陷阱 2-5**：单链表只有 head 时 `insert_last` 是 O(n)。PS1 的 1–4 题（Doubly Linked List，44 分）就是要求你实现双向链表并补齐 `delete_last` 等操作。
- **陷阱 2-6**：Python 的 `list.insert(0, x)` 是 O(n) 的 memmove，不能用它当队列；队列用 `collections.deque`（双端队列，两端摊还 O(1)）。

第 3 章 排序（L03）

3.1 问题

输入：n 个数的（静态）数组 A；输出：A 的一个**排序排列** B，满足 $B[i-1] \leqslant B[i]$。两个术语必须分清：

- **破坏性（destructive）**：排序覆写 A，而不是另建 B。
- **原地（in place）**：只用 O(1) 额外空间。原地 $\subseteq$ 破坏性。
- **稳定（stable）**：相等 key 的元素在输出中保持输入的相对顺序（第 5 章会用到）。

为什么排序重要？因为「有序数组」可以直接当 Set 用：`find` 用二分 O(log n)，`find_min` / `find_max` 是 Θ(1)，`find_next` / `find_prev` 是 O(log n)。讲义的对照表：

Set 实现	build(X)	find(k)	insert(x) / delete(k)	find_min / find_max	find_prev / find_next
Array（无序）	n	n	n	n	n
Sorted Array	n log n	log n	n	1	log n

3.2 接口视角

排序是「把 Set 接口的 build 做快」的工具。反过来说，**任何 Set 数据结构都定义了一个排序算法**：build（或逐个 insert）之后 iter_ord。第 7 章的 AVL Sort、第 8 章的 Heap Sort 都是这条原则的产物。讲义 L07 结尾专门点出这一点。

3.3 三种 $O(n^2)$ / $O(n \log n)$ 排序

先说一个反例：**排列枚举排序（Permutation Sort）**枚举 n! 个排列、逐个检查是否有序，时间 Ω(n!·n)，指数级，只用来说明「暴力正确但无用」。

选择排序（Selection Sort）

不变式：每轮在 A[:i+1] 里找最大元，换到 A[i]，然后递归排序 A[:i]。教材版伪代码：

```
SELECTION-SORT(A, n)
1  for i = n-1 downto 1
2      j = argmax_{0 <= k <= i} A[k]
3      exchange A[i] with A[j]
```

- 递推：$T(n) = T(n-1) + \Theta(n)$（找最大是 $\Theta(n)$）$\Rightarrow T(n) = \Theta(n^2)$。
- 交换次数是 O(n)——这是它唯一的优势（当「写」比「读」贵时有用，比如写 Flash）。
- **不稳定**：把最大元换到末尾会跨过相等元素。

插入排序（Insertion Sort）

不变式：先递归排序 A[:i]，再把 A[i] 通过相邻交换「冒」到正确位置。

```
INSERTION-SORT(A, n)
1  for i = 1 to n-1
2      j = i
3      while j > 0 and A[j] < A[j-1]
4          exchange A[j] with A[j-1]
5          j = j - 1
```

- 递推：$T(n) = T(n-1) + \Theta(n) \Rightarrow \Theta(n^2)$。
- **稳定**（只在严格小于时交换）、**原地**、**自适应**：对「k-近似有序」的输入是 $O(nk)$（讲义表格里的注释）。
- 最好的情况（已有序）是 $\Theta(n)$，这一点在选择排序里不存在。

归并排序（Merge Sort）

分成两半，各自排序，再用**双指针（two finger）**合并。

```
MERGE-SORT(A, a, b)
1  if b - a <= 1: return
2  c = floor((a+b)/2)
3  MERGE-SORT(A, a, c)
4  MERGE-SORT(A, c, b)
5  MERGE(A, a, c, b)

MERGE(A, a, c, b)          # A[a:c] 与 A[c:b] 各自有序
1  L = A[a:c]; R = A[c:b]  # θ(n) 额外空间
2  i = j = 0
3  for k = a to b-1
4      if j >= len(R) or (i < len(L) and L[i] <= R[j])
5          A[k] = L[i]; i += 1        # <= 保证稳定
6      else
7          A[k] = R[j]; j += 1
```

- 递推：$T(n) = 2T(n/2) + \Theta(n) \Rightarrow \Theta(n \log n)$（递归树：第 i 层 2^i 个结点、每个 n/2^i 工作量，每层合计 n，共 $\log_2 n$ 层）。
- **稳定**、**非原地**（$\Theta(n)$ 辅助空间）、比较模型下**最优**（第 5 章证下界）。

3.4 复杂度汇总

一句话结论：选择排序 $\Theta(n^2)$/原地/不稳定、插入排序 $\Theta(n^2)$/原地/稳定/对近似有序输入 $O(nk)$、归并排序 $\Theta(n \log n)$/非原地/稳定/比较模型最优；排列枚举是指数级反例。完整对照表（含计数、基数、快排、Timsort）见第 21.1 节。

3.5 实现陷阱

- **陷阱 3-1**：合并时用 `<` 而不是 `<=` 会破坏稳定性（相等时取了右边）。
- **陷阱 3-2**：`c = (a+b)//2` 与 `c = (a+b+1)//2` 都能工作，但必须保证两个子区间都非空（b−a > 1 的前提），否则无限递归。
- **陷阱 3-3**：每层都 `A[a:c]` 切片会分配 $\Theta(n)$ 内存，总分配量 $O(n \log n)$，常数很大。工程上用一个全局辅助数组 + 下标传递（推断：CPython 的 Timsort 用的是临时数组 + 插入排序小块）。
- **陷阱 3-4**：递归版插入排序在 n 大时会爆栈（Python 默认 1000 层），生产代码写循环版。

- **陷阱 3–5**：Python 的 `sorted()` 是 Timsort：稳定、最坏 O(n log n)、对部分有序数据接近 O(n)。不要手写归并去替代它，除非你要练习。
- **陷阱 3–6**：比较函数有副作用或不可传递（比如浮点 NaN、混合类型）会让任何排序的行为未定义。

第 4 章 哈希（L04）

4.1 问题：find 能不能比 O(log n) 更快

在**比较模型（comparison model）**下不能。论证是决策树（decision tree）：

- 任何比较算法是一棵二叉树，内部结点是一次二元比较，叶子是一个输出；
- 一次执行 = 一条根到叶的路径，运行时间 ⩾ 路径长度 ⩾ 树高；
- 搜索需要 ⩾ n+1 个叶子（n 个「找到第 i 个」+ 1 个「没找到」）；
- 二叉树高 ⩾ ⌈lg(n+1)⌉ − 1 = Ω(log n)。

所以有序数组已经达到比较模型的下界。要更快，必须引入**分支因子 ω(1) 的操作**——也就是 Word-RAM 的 O(1) 随机访问。

4.2 直接寻址数组（Direct Access Array）

给每个元素一个唯一整数 key k ∈ {0,…,u−1}，直接存到下标 k。若 u ⩽ 2^w（key 能放进一个字），`find` / `insert` / `delete` 都是**最坏 O(1)**。代价是空间 Θ(u)：讲义举例，10 个字母的名字若一位一比特需要 26^10 ≈ 17.6 TB。

4.3 哈希表与冲突

把 u 压到 m = Θ(n)：哈希函数 h(k): {0,…,u−1} → {0,…,m−1}。若 m < u，鸽巢原理保证**不存在单射**，必有冲突（collision）。两种处理：

- **开放寻址（open addressing）**：冲突时在表内另找位置。分析复杂但工程常用。
- **链地址法（chaining）**：每个槽挂一个支持动态 Set 接口的容器（链表、动态数组、甚至另一张哈希表）。

链地址法下，若 key 大致均匀分布，链长 = n/m = O(1)，所有操作 O(1) 期望。但**固定哈希函数一定存在坏输入**：取 h(k) = 常数，链长 Θ(n)。

4.4 全域哈希（Universal Hashing）

讲义给的族：

$$h_{ab}(k) = (((ak + b) \bmod p) \bmod m)$$

其中 p > u 是固定素数，a,b ∈ {0,…,p−1} 且 a ≠ 0。它满足**全域性**：对任意 k_i ≠ k_j，Pr{h(k_i) = h(k_j)} ⩽ 1/m。

期望链长推导（讲义原文思路）：

- 指示变量 X_ij = 1 当且仅当 h(k_i) = h(k_j)；
- 槽 h(k_i) 的链长 X_i = Σ_j X_ij；
- E[X_i] = 1 + Σ_{j≠i} Pr{h(k_i)=h(k_j)} ⩽ 1 + (n−1)/m；
- 由于 m = Ω(n)，负载因子 α = n/m = O(1)，故期望 O(1)。

关键点：**期望是对随机选择的哈希函数取的，与输入无关**（input-independent），所以对手无法构造坏输入。

动态性：当 n/m 远离 1 时，重新选一个随机哈希函数并重建表；摊还分析与动态数组相同。于是哈希表实现动态 Set 操作是**期望摊还 O(1)**。

4.5 复杂度表（讲义原文）

数据结构	build(X)	find(k)	insert(x)	delete(k)	find_min / find_max	find_prev / find_next
Array	n	n	n	n	n	n
Sorted Array	n log n	log n	n	n	1	log n
Direct Access Array	u	1	1	1	u	u
Hash Table	n(e)	1(e)	1(a)(e)	1(a)(e)	n	n

`(e)` = 期望，`(a)` = 摊还。注意 find_min/find_max/find_prev/find_next 在哈希表上是 Θ(n)：**哈希表不支持有序操作**，这是它与平衡树的分界线。

4.6 实现骨架

```
class ChainedHashTable:
    def __init__(self, m=8):
        self.m, self.n, self.T = m, 0, [[] for _ in range(m)]
        self.p = (1 << 61) - 1
        self._reseed()
    def _reseed(self):
        import random
        self.a, self.b = random.randrange(1, self.p), random.randrange(self.p)
    def _h(self, k): return ((self.a * k + self.b) % self.p) % self.m
    def find(self, k):
        for kk, v in self.T[self._h(k)]:
            if kk == k: return v
        return None
    def insert(self, k, v):
        i = self._h(k)
        for j, (kk, _) in enumerate(self.T[i]):
            if kk == k: self.T[i][j] = (k, v); return   # 覆盖语义
        self.T[i].append((k, v)); self.n += 1
        if self.n > 2 * self.m: self._rebuild(2 * self.m)   # 负载因子 > 2 重建
```

`delete` 需要先 `find` 定位再删除（链是 list，删除是 O(链长)）。

4.7 实现陷阱

- **陷阱 4–1**：key 必须可哈希且**不可变**。Python 里 list 不能当 dict 的 key，因为 `__hash__` 与 `__eq__` 不一致会导致「存进去找不到」。
- **陷阱 4–2**：自定义类要同时实现 `__hash__` 和 `__eq__`，且相等对象必须有相同 hash。只实现 `__eq__` 会让 Python 把 `__hash__` 置为 None（不可哈希）。
- **陷阱 4–3**：`m` 取 2 的幂且用 `h(k) = k mod m` 时，只用到 key 的低位。讲义明确说除法哈希要「避开 key 的对称性」，大素数比 2 的幂安全。
- **陷阱 4–4**：固定哈希函数 + 攻击者可控 key = 哈希洪泛（hash flooding），退化成 O(n) 链表。Python 3.3+ 默认对 str 做哈希随机化（PYTHONHASHSEED）就是为了防这个（工程补充，非 OCW 内容）。
- **陷阱 4–5**：开放寻址的删除不能直接置空，否则查找链断裂；要用 tombstone（墓碑）标记，且墓碑多了要重建（工程补充）。
- **陷阱 4–6**：把哈希表当有序容器用。要 `min` / `max` /前驱后继，请用第 7 章的 AVL 或第 8 章的堆。

第 5 章 线性排序：计数排序与基数排序（L05）

5.1 问题：能不能突破 n log n

比较排序的下界来自信息论：n 个元素有 n! 种排列，决策树至少 n! 个叶子，高 ≥ log(n!)。

$$\log(n!) \geq \log\left((n/2)^{n/2}\right) = \Omega(n \log n)$$

所以归并排序在比较模型下是最优的。突破口还是**直接寻址**：用 key 当下标，分支因子从 2 变成 u。

5.2 直接寻址排序（Direct Access Array Sort）

前提：key 是 {0,…,u−1} 内**互不相同**的非负整数，n ⩽ u。做法：建长 u 的数组，把元素放到下标 key 处，再顺序读出。

```
def direct_access_sort(A):
    D = [None] * (1 + max(x.key for x in A))      # Θ(u)
    for x in A: D[x.key] = x                      # key 必须唯一
    A[:] = [x for x in D if x is not None]        # 按槽序读出
```

时间 Θ(n + u)：u = Θ(n) 时是线性的。**但它不能处理重复 key**（后写覆盖先写），也不稳定。

5.3 元组排序与稳定性

若 u 很大（比如 u = Ω(n²)），把 key 写成 n 进制两位数：k = a·n + b，其中 a = ⌊k/n⌋ < n、b = k mod n（Python 里就是 `divmod(k, n)`）。排序元组要按**字典序**，做法是**从最不显著位到最显著位**依次排序（least significant digit first）。

为什么必须从低位到高位？因为高位排序会打乱低位顺序；而低位排序的成果要靠**稳定性**在高位排序后保留下来。这就是基数排序（radix sort）的全部秘密。

计数排序（Counting Sort）：直接寻址排序的稳定版。每个槽挂一条链，插入时 `append` 到链尾（保持输入顺序），读出时按槽序、链序输出。

```
def counting_sort(A, key=lambda x: x.key):
    D = [[] for _ in range(1 + max(key(x) for x in A))]
    for x in A: D[key(x)].append(x)               # append 到链尾 => 稳定
    A[:] = [x for chain in D for x in chain]
```

时间 Θ(n + u)，空间 Θ(n + u)，**稳定**，非原地。

> 工程补充：教科书的计数排序常写成「前缀和 + 从右往左填」的版本，它不需要链，空间 Θ(n+u) 但常数更小。两种写法等价，讲义给的是链版本，因为它直接复用了第 4 章的哈希冲突处理思路。

5.4 基数排序（Radix Sort）

若每个 key < n^c（c 为常数），则每个 key 是 c 位 n 进制数，写成 c 元组，对每一位做一次计数排序：

```
def radix_sort(A):
    n = len(A); u = 1 + max(x.key for x in A)
    c = 1 + (u.bit_length() // n.bit_length())   # n 进制位数
    digits = [None] * n                          # digits[i] = key 的 c 位（低位在前）
    for i, x in enumerate(A):
        D, high = [], x.key
        for _ in range(c):
            high, low = divmod(high, n); D.append(low)
        digits[i] = D
    pairs = list(zip(digits, A))                 # 位向量必须与元素一起搬动
    for j in range(c):                           # 低位 -> 高位，每轮稳定排序
        bucket = [[] for _ in range(n)]
        for D, x in pairs: bucket[D[j]].append((D, x))
        pairs = [p for b in bucket for p in b]   # 桶内保持输入顺序 => 稳定
    A[:] = [x for _, x in pairs]
```

- 时间：c 轮 × O(n) = **O(c·n)**；c = O(log_n u)，所以是 O(n + n log_n u)。
- 当 u = O(n^c)（c 常数）时是 **O(n)**，线性！
- **稳定**、非原地。
- 注意 `c = 1 + (u.bit_length() // n.bit_length())` 这个写法：它用位长之比近似 ⌈log_n u⌉，是讲义原文的取法（可能多算一位，不影响渐进）。

5.5 实现陷阱

- **陷阱 5–1**：基数排序必须从低位到高位，且每轮排序必须稳定。两条件缺一不可。
- **陷阱 5–2**：`u = max(key)+1` 在 key 含负数时直接崩（负下标）。正确做法是先偏移 `key - min_key`，或对符号位单独处理。
- **陷阱 5–3**：key 是浮点时不能直接当数组下标。要么量化成整数，要么退回比较排序。
- **陷阱 5–4**：u ≫ n 时（比如 key 是 64 位随机数）计数排序的 Θ(u) 空间会爆。此时基数排序按位分组，或直接用比较排序。
- **陷阱 5–5**：把「线性时间」当成「更快」。n = 1000 时基数排序的常数（c 轮 × 建数组）常常打不过 Timsort；线性只在 n 很大且 key 分布合适时兑现。
- **陷阱 5–6**：计数排序的链版本用 `append` 到 list 尾部，每个槽一个 list 对象，n 大时对象开销显著（工程补充：可用单个数组 + 计数 + 前缀和替代）。

第 6 章 二叉树（L06）

6.1 问题：把 O(h) 变成 O(log n)

前四章的最好成绩是哈希表的期望 O(1)，但它不支持有序操作。有序操作（min/max/前驱/后继）目前只有有序数组能做，而有序数组的插入是 O(n)。目标是找到一个数据结构，让 Set 与 Sequence 的所有操作都是 **O(log n) 最坏**（不是期望）。

目标（讲义原表）：Set 的 build 为 n log n，find/insert/delete/min/max/前驱后继全部 log n；Sequence 的 build 为 n，get_at/set_at/insert_at/delete_at 全部 log n。

6.2 结构：三个指针的结点

结点表示 `node.{item, parent, left, right}`。术语：

- **根（root）**：没有 parent；
- **叶（leaf）**：没有孩子；
- **深度 depth(X)**：X 到根的路径长度；
- **高度 height(X)**：以 X 为根的子树中最大的深度。

核心思想：**所有操作做到 O(h)，再让 h = O(log n)**（第 7 章负责后半句）。

二叉树有一个**内蕴的顺序**，叫遍历序（traversal order）：左子树全部在 X 之前，X 在右子树全部之前。递归地「左 → 自己 → 右」就是中序遍历，时间 O(n)。

6.3 导航操作（都是 O(h)）

- **first(X)**：X 子树中遍历序的第一个结点——有左孩子就往左走到底，否则 X 自己。
- **last(X)**：对称。
- **successor(X)**：有右孩子 → 右子树的 first；否则往上找「X 在其左子树中」的最低祖先。
- **predecessor(X)**：对称。

6.4 动态操作（只增删叶子）

- **insert_after(X, Y)**：若 X 无右孩子，Y 成为 X 的右孩子；否则 Y 成为 X 的后继的左孩子（后继一定没有左孩子）。
- **insert_before(X, Y)**：对称。
- **delete(X)**：若 X 是叶子，直接摘掉；否则与 predecessor（若有左孩子）或 successor（否则）交换 item 后递归，把待删元素「换到叶子再摘」。

6.5 两种应用：Set 与 Sequence

应用一：Set = BST

让**遍历序 = key 递增序**，等价于 **BST 性质**：每个结点左子树的所有 key ⩽ 该结点 key ⩽ 右子树所有 key。于是 `find(k)` 就是二分。

```
class Node:
    __slots__ = ("item", "parent", "left", "right", "size")
    def __init__(self, item):
        self.item, self.parent, self.left, self.right, self.size = item, None, None, None,

class BST:
    def __init__(self): self.root, self.n = None, 0
    def find(self, k):                          # O(h)
        x = self.root
        while x and x.item != k: x = x.left if k < x.item else x.right
        return x
    def insert(self, k):                        # O(h)，并在沿路维护 size
        y, x = None, self.root
        while x:
            y = x; x.size += 1
            if k == x.item: return
            x = x.left if k < x.item else x.right
        z = Node(k); z.parent = y
        if y is None: self.root = z
        elif k < y.item: y.left = z
        else: y.right = z
        self.n += 1
```

应用二：Sequence = 带 size 增广的树

让**遍历序 = 序列顺序**。要支持 `get_at(i)`，需要知道「左子树有多少个结点」，于是给每个结点加 `size` 字段：

```
subtree_at(X, i)                        # 返回 X 子树中第 i 个（0-based）
1  if X is None: raise IndexError
2  nL = size(X.left)                    # 空子树为 0
3  if i < nL:      return subtree_at(X.left, i)
4  elif i == nL:   return X
5  else:           return subtree_at(X.right, i - nL - 1)
```

- 插入一个叶子时，沿祖先链 +1，O(h)；删除时 −1，O(h)。
- 有了 O(1) 的 `size`，「第 i 个」变成 O(h)。
- 朴素的 `build` 是 O(nh)，但可以做到 O(n)（复习课内容）。

这就是**增广（augmentation）**的第一次出现，第 7 章会把它系统化。

6.6 复杂度

操作	二叉树	目标
build	n log n（逐个插）	n log n
find / insert / delete	h	log n
find_min / find_max / next / prev	h	log n
get_at / set_at / insert_at / delete_at	h	log n

6.7 实现陷阱

- **陷阱 6-1**：`delete` 时直接摘掉有两个孩子的结点，会丢掉整棵子树。必须换到叶子再摘。
- **陷阱 6-2**：`size` 忘了在旋转后更新（第 7 章）或忘了在删除路径上更新，`subtree_at` 就会越界或返回错元素。
- **陷阱 6-3**：BST 性质用严格小于还是小于等于，决定了能否存重复 key。6.006 的 Set 接口假设 key 唯一；要存重复 key，用「结点存一个序列」或第 8 章的多重集方案。
- **陷阱 6-4**：按有序序列逐个 `insert` 会退化成一条链，h = n，所有操作 O(n)。这正是第 7 章要修的。
- **陷阱 6-5**：`parent` 指针维护不全，`successor` 会走错。建议每次改指针后立刻检查三处：child.parent、parent.child、root。
- **陷阱 6-6**：用递归实现 `subtree_at` 时，h 大的退化树会爆栈。

第 7 章 AVL 树与增广（L07）

7.1 问题：让 h = O(log n)

平衡（balanced）：在动态操作下维持 h = O(log n) 的二叉树。方案很多（红黑树、splay、2-3 树……），历史上第一个是 **AVL 树**（Adelson-Velsky 与 Landis，1962——讲义原文标注的年份）。

7.2 旋转：改结构但不改遍历序

旋转（rotation）只重连 O(1) 个指针，保持中序遍历不变。左旋/右旋互为逆操作。

```
rotate_left(X)                    # X 的右孩子 Y 上位
1  Y = X.right
2  X.right = Y.left
3  if Y.left: Y.left.parent = X
4  Y.parent = X.parent
5  if X.parent is None: T.root = Y
6  elif X is X.parent.left: X.parent.left = Y
7  else: X.parent.right = Y
8  Y.left = X; X.parent = Y
9  update(X); update(Y)           # 先 X 后 Y：Y 的高度依赖 X
```

讲义的一个关键引理：**O(n) 次旋转可以把任意二叉树变成同遍历序的任意另一棵**（反复做「最靠后的右旋」得到规范链，每次让最后结点深度 +1，最终深度 n−1，故至多 n−1 次）。这说明「用旋转维护平衡」在原理上可行，只是不能每次都用 O(n) 次。

7.3 平衡条件与高度界

- **高度平衡（height-balanced）**：结点的左右子树高度差 ⩽ 1。
- **偏斜（skew）**：右子树高度 − 左子树高度，取值 −1/0/1 时平衡。
- 引理：高度平衡树的 h = O(log n)。证明用「高度 h 的平衡树最少结点数」F(h)：F(0)=1、F(1)=2、F(h) = 1 + F(h−1) + F(h−2) ⩾ 2F(h−2) ⇒ F(h) ⩾ 2^{h/2}，即 n ⩾ 2^{h/2}，故 h ⩽ 2 $\log_2$ n。

7.4 局部再平衡（4 种情形）

设最低的不平衡结点为 B，skew(B) = 2（对称情形取负）：

情形	孩子 F 的 skew	操作	结果
1（LL 型，镜像）	skew(F) = 0	对 B 左旋	B、F 均平衡，高度不变
2（RR 型）	skew(F) = 1	对 B 左旋	B 的 skew 变 0，F 变 −1，高度减 1
3（RL 型）	skew(F) = −1	先对 F 右旋，再对 B 左旋	B、F、D 均平衡，高度减 1

讲义强调的细节：局部再平衡后 **B 的高度要么不变（情形 1）要么减 1（情形 2、3）**。这个「高度是否减 1」直接决定了插入和删除的后续行为。

7.5 全局再平衡

- **插入**：只有祖先链上的结点可能失衡，最低失衡点必落在情形 2 或 3，一次（单或双）旋转就恢复平衡，且子树高度回到原值 ⇒ **插入最多 1 次旋转**。

- **删除**：最低失衡点可能落在情形 1（高度不变）或 2/3（高度减 1）。若高度减 1，父结点可能继续失衡 ⇒ **删除可能要沿祖先链做 O(log n) 次旋转**。

这就是「AVL 插入快、删除稍慢」的根源，也是红黑树存在的动机（红黑树把删除的旋转次数压到 O(1)）。

7.6 增广（augmentation）的系统方法

要在二叉树上维护子树属性 P，只需两步：

1. 说明 P(X) 是什么；
2. 说明如何用孩子结点的 P 在 O(1) 内算出 P(X)。

于是动态操作的代价不变（每次改结构时顺路更新 O(1) 个结点 + 沿途 O(h) 个祖先）。AVL 用它维护两个字段：

- `height`：`1 + max(height(left), height(right))`，用于算 skew；
- `size`：`1 + size(left) + size(right)`，用于 Sequence 的 `get_at`。

7.7 插入/删除的实现骨架

```
class AVL:
    def _h(self, x): return x.height if x else -1
    def _sz(self, x): return x.size if x else 0
    def _update(self, x):
        x.height = 1 + max(self._h(x.left), self._h(x.right))
        x.size = 1 + self._sz(x.left) + self._sz(x.right)
    def _skew(self, x): return self._h(x.right) - self._h(x.left)
    def _rebalance(self, x):                        # 返回子树新根
        self._update(x)
        if self._skew(x) == 2:
            if self._skew(x.right) < 0: self._rotate_right(x.right)
            return self._rotate_left(x)
        if self._skew(x) == -2:
            if self._skew(x.left) > 0: self._rotate_left(x.left)
            return self._rotate_right(x)
        return x
    # insert: BST 插叶 -> 从父结点向上 while x: x = self._rebalance(x); x = x.parent
    # delete: 换到叶子摘掉 -> 同样一路 _rebalance 到根（不能提前停）
```

删除是「换到叶子 → 摘掉 → 从父结点自底向上 `_rebalance`」。**必须一路修到根**，不能像插入那样提前停。

7.8 复杂度与用途

一句话结论：Set AVL 全部操作 O(log n)（build O(n log n)、iter O(n)）；Sequence AVL 的 get_at/insert_at/delete_at 都是 O(log n)（build O(n)、iter O(n)）。AVL Sort = build + iter_ord = O(n log

n)；用「子树 max/min 指针」增广后，AVL 可以直接当优先队列（find O(1)、delete O(log n)、build O(n)）。

7.9 实现陷阱

- **陷阱 7-1**：旋转后更新顺序错。`rotate_left(X)` 必须先 `_update(X)` 再 `_update(Y)`，因为 Y 的新高度依赖 X 的新高度；反过来会算出过期高度。
- **陷阱 7-2**：插入时「修到第一个平衡的祖先就停」是错的通用写法。虽然插入的旋转只发生一次，但**高度更新必须一路到根**（否则祖先的 height 字段是脏的）。
- **陷阱 7-3**：删除只修一层就返回，导致上层残留 skew = ±2。删除必须 `while x: x = rebalance(x); x = x.parent`。
- **陷阱 7-4**：判断情形时用「孩子的高度」而不是「孩子的 skew」。用 skew 更省事，但要注意空孩子的 skew 定义（空树高度 −1）。
- **陷阱 7-5**：`_rebalance` 里返回新的子树根，但忘了把它接回父结点，树就断了。
- **陷阱 7-6**：同时维护 height 与 size 时，`_update` 要一次算完两个字段，分两处写容易漏。
- **陷阱 7-7**：Python 没有内置平衡树。工程上需要有序容器时，常用 `sortedcontainers.SortedList`（块状分解，不是 AVL）或 `bisect` + list（插入 O(n)）。知道「标准库没有 AVL」这件事本身很重要。

第 8 章 二叉堆与堆排序（L08）

8.1 问题：只要最大/最小，能不能更简单

优先队列（priority queue）只关心「最重要的那个」：`build`、`insert`、`delete_max`、`find_max`（或 min）。应用：路由器带宽调度、操作系统进程调度、离散事件模拟、图算法（Dijkstra 的 PQ）。

任何优先队列都定义了一个排序算法：build 后反复 `delete_max`。四种实现的对照（讲义原表，完整表见第 20 章）：

无序动态数组 = 选择排序（build n、delete_max n）；有序数组 = 插入排序（build n log n、insert n）；AVL = AVL Sort（全部 log n、非原地）；二叉堆 = Heap Sort（build n、insert/delete log n、**原地**、n log n）。

8.2 结构：数组即完全二叉树

完全二叉树（complete binary tree）与数组之间有双射：按「根到叶、左到右」的读序填充。于是不需要指针，用下标算术：

```
left(i)   = 2i + 1
right(i)  = 2i + 2
parent(i) = (i - 1) // 2
```

n 个元素的完全二叉树高度是 ⌈lg n⌉，天然平衡。

最大堆性质（max-heap property）：对每个 i，Q[i] ⩾ Q[left(i)] 且 Q[i] ⩾ Q[right(i)]。由归纳可证：堆中每个结点都 ⩾ 其子树中所有结点，因此**最大值在根**。

8.3 两个核心操作

```
max_heapify_up(Q, i)
1  while i > 0 and Q[parent(i)] < Q[i]
2      swap Q[i], Q[parent(i)]
3      i = parent(i)

max_heapify_down(Q, i, n)
1  while True
2      l, r = left(i), right(i); largest = i
3      if l < n and Q[l] > Q[largest]: largest = l
4      if r < n and Q[r] > Q[largest]: largest = r
5      if largest == i: break
6      swap Q[i], Q[largest]
7      i = largest
```

- `insert(x)`：`append` 到末尾（摊还 O(1)），再 `heapify_up`，O(log n)。
- `delete_max()`：把根与末尾交换，弹出末尾，再对根 `heapify_down`，O(log n)。

8.4 线性建堆（build heap in O(n)）

- 逐个 `insert`（自顶向下）代价 ≈ Σ depth(i) = lg(n!) = Ω(n log n)。
- **自底向上**：先把整个数组当完全二叉树，然后从 i = n−1 递减到 0 做 `heapify_down`。代价 ≈ Σ height(i) = Σ (lg n − lg i) = lg(n^n / n!) = Θ(n)。

```
def build_max_heap(A):
    for i in range((len(A) - 2) // 2, -1, -1):    # 最后一个非叶结点起，自底向上
        sift_down(A, i, len(A))
```

直觉：绝大多数结点在底层，它们的高度是 0 或 1，几乎不用下沉；只有极少数高层结点需要走远。

8.5 堆排序（原地）

把最大堆当作大数组的一个前缀：`|Q|` 从 0 涨到 n（插入阶段），再从 n 降到 0（删除阶段）。

```
def heap_sort(A):
    build_max_heap(A)                              # O(n)
    for end in range(len(A) - 1, 0, -1):
        A[0], A[end] = A[end], A[0]                # 最大值归位
        sift_down(A, 0, end)                       # 在 [0, end) 内恢复堆序
```

- 时间 O(n log n)（建堆 O(n) + n−1 次 O(log n) 下沉）。
- **原地**（只用 O(1) 额外空间）、**不稳定**。

8.6 实现陷阱

- **陷阱 8-1**：0-index 与 1-index 的公式不同。CLRS 用 1-index（left = 2i，right = 2i+1，parent = ⌊i/2⌋），Python 用 0-index（left = 2i+1）。混用必错。
- **陷阱 8-2**：`build` 从 `n//2` 开始是错的。最后一个非叶结点下标是 `(n-2)//2`；从 `n//2` 开始会漏掉一个内部结点。
- **陷阱 8-3**：`delete_max` 先 `heapify_down` 再交换，会把错误元素放到末尾。
- **陷阱 8-4**：`sift_down` 必须与**较大的那个孩子**交换；与任意孩子交换会破坏堆序。
- **陷阱 8-5**：堆排序不稳定。需要稳定 + O(n log n) 时用归并排序。
- **陷阱 8-6**：堆只保证「局部有序」，中序遍历不是有序序列。要全序请用 BST/AVL。
- **陷阱 8-7**：Python 的 `heapq` 是**最小堆**。要最大堆就存 `-x`，或存 `(priority, seq, item)` 元组（`seq` 用来打破不可比较对象的平局）。
- **陷阱 8-8**：`heapq` 不提供 `decrease_key`。Dijkstra 里要用「懒删除」（第 13 章），不要试图在堆里找元素。

第 9 章 广度优先搜索（L09）

9.1 问题

图 G = (V, E)：V 是顶点集，E ⊆ V×V 是边集。本课假设图是**简单图**：边不重复、无自环，于是 $|E| = O(|V|^2)$。

需要解决的问题层级（讲义原文）：

问题	输入	输出
单对可达（Single Pair Reachability）	G, s, t	是否存在 s→t 的路径
单源可达（Single Source Reachability）	G, s	所有从 s 可达的顶点
单对最短路（Single Pair Shortest Path）	G, s, t	δ(s,t) 与一条最短路

问题	输入	输出
单源最短路（SSSP）	G, s	所有 δ(s,v) 与最短路树

下面的问题至少和上面的问题一样难（可以用解下面的黑盒来解上面的），所以只要解决最难的 SSSP，其他都免费。

9.2 表示：邻接表（adjacency list）

图要存的是每个顶点 u 的出邻居集合 $Adj^+(u)$。两层结构：

- 外层 `Adj`：把顶点映射到它的邻接表，用**直接寻址数组**（顶点标号 0…|V|−1）或**哈希表**（任意标号）；
- 内层 `Adj(u)`：用**数组或链表**（通常只需要迭代）。

空间 Θ(|V| + |E|)（由握手引理 Σ deg(u) = 2|E|）。所以**图算法的「线性时间」指 Θ(|V| + |E|)**，这点必须记牢。

三种典型写法（讲义例）：

```
G1 = [[2, 1], [2, 0, 3], [1, 3, 0], [1, 2]]          # 无向，顶点 0..3
G2 = [[2], [2, 0], [1]]                              # 有向
G3 = {"a": ["s", "b"], "b": ["a"], "s": ["a", "c"],
      "c": ["s", "d", "e"], "d": ["c", "e", "f"],
      "e": ["c", "d", "f"], "f": ["d", "e"], "g": []}  # 无向，字符串标号
```

加权图两种存法：把权重塞进邻接表的元素（`(v, w)` 元组），或用单独的 Set 把「边」映射到权重。讲义假设**查一条边的权重是 O(1)**。

9.3 路径与距离

- **路径**是顶点序列 $(v_1,\dots,v_k)$ 且相邻有边；**简单路径**不重复顶点；
- **长度**是边数；**距离 δ(u,v)** 是最短路径长度，不连通时为 ∞；
- 全部路径可能长达 Ω(|V|)，逐条返回要 $\Omega(|V|^2)$ 时间，所以只存**父指针 P(v)**（最短路上的前驱），构成 **O(|V|) 的最短路树**。

9.4 BFS：按距离分层

记 L_i = {v | δ(s,v) = i}。关键引理：L_i 中每个顶点必与 L_{i−1} 中某顶点相邻；且 L_j (j<i) 中的顶点不会出现在 L_i。于是可以逐层扩张。

```
BFS(G, s)
1  for v in V: d[v] = INF; P[v] = None
2  d[s] = 0; Q = [s]
3  while Q not empty:
4      u = Q.popleft()
5      for v in Adj(u):
6          if d[v] == INF:            # 未访问
7              d[v] = d[u] + 1
8              P[v] = u
9              Q.append(v)
```

```
def bfs(adj, s):
    d, P, Q = {s: 0}, {s: None}, deque([s])
    while Q:
        u = Q.popleft()
        for v in adj[u]:
            if v not in d:                          # 入队时标记，天然去重
                d[v], P[v] = d[u] + 1, u; Q.append(v)
    return d, P                                     # 不在 d 中 => 不可达
```

9.5 复杂度

- 每个顶点入队至多一次，每条边被检查一次：**O(|V| + |E|)**。
- 空间：队列 O(|V|) + 距离表 O(|V|)。

9.6 实现陷阱

- **陷阱 9–1**：出队时才标记 visited 会让同一顶点入队多次，复杂度退化。必须在**入队时**标记。
- **陷阱 9–2**：用 list 当队列（`pop(0)`）是 O(n)，整体退化成 $O(V^2)$。用 `deque`。
- **陷阱 9–3**：BFS 只在**无权**（或所有边权相同）时给最短路。加权图要用第 11–13 章的算法。
- **陷阱 9–4**：要求输出「字典序最小」的访问顺序时（PS5 的 5–1(c) 就要求按字母序访问邻居），必须先把邻接表排序。
- **陷阱 9–5**：只跑一次 BFS 得到的是「从 s 可达的分量」，不是全图的连通分量。全图要 **Full-BFS**（对每个未访问点再跑一次）。

第 10 章 深度优先搜索、拓扑排序与环检测（L10）

10.1 问题

DFS 解的是**单源可达**，不保证最短。它的价值在于**完成顺序（finishing order）**携带的结构信息：拓扑排序、环检测都靠它。

10.2 算法

```
visit(u)
1  for v in Adj(u):
2      if v not in P:
3          P[v] = u
4          visit(v)
5  # 此处 u 完成（finish）

DFS(s): P[s] = None; visit(s)
```

Full-DFS：对所有未访问顶点依次调用 `visit`，得到**森林**。

- 单次 DFS：O(|E|)（不维护距离表）；
- Full-DFS：O(|V| + |E|)。

10.3 三个应用

连通分量（Connected Components）

无向图：Full-DFS 每启动一次 `visit` 就得到一个连通分量。判定连通只需看分量数是否为 1。

拓扑排序（Topological Sort）

DAG（有向无环图）是含无有向环的有向图。**拓扑序**是顶点的一个排列 f，使每条边 (u,v) 满足 f(u) < f(v)。

引理（讲义原文）：若 G 是 DAG，则**完成顺序的逆序**是拓扑序。证明分两种情形：

- 若 u 比 v 先被访问：在 u 完成之前一定会访问到 v（沿 (u,v) 或别的路径），所以 v 先完成；
- 若 v 比 u 先被访问：因为无环，u 不可能从 v 到达，所以 v 先完成。

两种情形都是「v 先完成」，故逆完成序中 u 在 v 之前。

```
def topo_sort(adj, V):
    state, order = {}, []                          # 0 未访问 / 1 栈上 / 2 完成
    def visit(u):
        state[u] = 1
        for v in adj[u]:
            if state.get(v, 0) == 0: visit(v)
            elif state[v] == 1: raise ValueError("cycle")   # 回边
        state[u] = 2; order.append(u)
    for u in V:
        if state.get(u, 0) == 0: visit(u)
    return order[::-1]                             # 逆后序 = 拓扑序
```

环检测（Cycle Detection）

若逆完成序不是拓扑序，则必有环。检测方法：对每条边 (u,v) 检查 v 是否在 u 之前，O(|E|)。要**返回**一个环，就在 DFS 中维护当前递归栈上的祖先集合：有环时必然遍历到一条「指向祖先」的边（回边/back edge）。

10.4 复杂度

任务	时间
单源 DFS	O(|E|)
Full-DFS / 连通分量 / 拓扑排序	O(|V| + |E|)
环检测（含返回环）	O(|V| + |E|)
验证给定顺序是否为拓扑序	O(|V| + |E|)

10.5 实现陷阱

- **陷阱 10-1**：Python 递归深度默认 1000。链状图（|V| > 1000）会 `RecursionError`，必须写迭代版（显式栈 + 迭代器状态）。
- **陷阱 10-2**：拓扑排序用「前序」（访问时 append）而不是「后序取反」，结果是错的。
- **陷阱 10-3**：拓扑序只对 DAG 存在。有环时算法不会报错，只是给出一个无意义的顺序——必须显式检测。
- **陷阱 10-4**：用「入度为 0 的队列」（Kahn 算法）也能拓扑排序，且天然检测环（处理完的顶点数 < |V| 即有环）。DFS 版与 Kahn 版都对，考试写哪个都行。
- **陷阱 10-5**：无向图 DFS 中「指向已访问邻居」的边不一定是环（可能是父边）。判断环要看是否指向**当前栈上的祖先**，不是「已访问」。
- **陷阱 10-6**：`state` 用 dict 的 `.get` 默认 0，容易把「已访问」误判为「未访问」。用 `collections.defaultdict(int)` 或显式初始化。

第 11 章 加权最短路与 DAG 松弛（L11）

11.1 问题

加权图：G = (V,E) 加权重函数 w: E → ℤ。路径权重是边权之和，δ(s,t) 是最小权重。

- 若无路径，δ = ∞；
- 若存在**负权环**且 s 能到达它、它又能到达 t，则 δ(s,t) = −∞（讲义强调这里用 infimum 而非 minimum，就是因为可能不存在有限的最小值）；
- 最短路的子路径也是最短路（否则可以把更短的一段接进去）。

11.2 松弛（relaxation）框架

维护**距离估计** d(s,v)，初值 ∞（源点 0），始终满足 d(s,v) ≥ δ(s,v) 且 d(s,v) 是某条路径的权重。

三角不等式：δ(u,v) ≤ δ(u,x) + δ(x,v)。当 d(s,v) > d(s,u) + w(u,v) 时违反不等式，就**松弛**边 (u,v)：d(s,v) ← d(s,u) + w(u,v)。

引理（松弛安全）：若每个 d 都是某条路径的权重，松弛后仍如此。整个最短路家族的正确性都建在这条引理上。

11.3 DAG 松弛

DAG 上按拓扑序处理每个顶点，松弛它的所有出边。

```
DAG-RELAXATION(G, s)
1  for v in V: d[v] = INF
2  d[s] = 0
3  for u in topological_order(G):        # O(|V|+|E|)
4      for (v, w) in Adj(u):
5          if d[v] > d[u] + w:
6              d[v] = d[u] + w
```

正确性：对拓扑序归纳。处理到 v 时，其在最短路上的前驱 u 已被处理且 d[u] = δ(s,u)，于是 d[v] 被压到 ≤ δ(s,v)；又由松弛安全 d[v] ≥ δ(s,v)，故相等。

复杂度：**O(|V| + |E|)**。

11.4 父指针的事后重建

讲义给了一个很实用的技巧：先只算距离，再在 O(|V|+|E|) 内重建最短路树。对每个有限距离的 u，检查出边 (u,v)：若 P(v) 未赋值且 d(v) = d(u)+w(u,v)，则 P(v) = u。零权环会导致父指针成环，需要额外标记/打断（讲义给了一段专门的循环处理）。**这就是为什么后续算法只关注距离**。

```
def dag_shortest_paths(adj, s, V):
    d = {v: float("inf") for v in V}; d[s] = 0
    plain = {u: [v for v, _ in adj[u]] for u in V}        # 拓扑排序只看结构
    for u in topo_sort(plain, V):                  # 拓扑序 => 一次松弛即定稿
        if d[u] == float("inf"): continue
        for v, w in adj[u]:
            if d[v] > d[u] + w: d[v] = d[u] + w
    return d
```

11.5 复杂度对照

图	权重	算法	时间
一般	无权	BFS	|V| + |E|
DAG	任意	DAG Relaxation	|V| + |E|
一般	任意	Bellman–Ford	|V|·|E|
一般	非负	Dijkstra	|V| log|V| + |E|

11.6 实现陷阱

- **陷阱 11–1**：拓扑序必须在松弛**之前**算好；在松弛过程中动态算序会读到未定的 d。
- **陷阱 11–2**：`float("inf")` 参与加法会变成 inf，逻辑上没问题，但 `inf + (-5)` 仍是 inf，于是「不可达」不会被误松弛——这是正确行为，不要改成 0。
- **陷阱 11–3**：用 `d[u] + w` 时没检查 d[u] 是否可达，可能把 inf 写进别的点（结果仍是 inf，但会浪费一次赋值；若用整数哨兵如 10^18 就要小心溢出）。
- **陷阱 11–4**：DAG 松弛要求图真的是 DAG。含环时会漏掉经由环的路径。
- **陷阱 11–5**：父指针重建时，零权环里的点互指成环，`while p[v]` 会死循环。

第 12 章 Bellman–Ford（L12）

12.1 问题

一般图 + 任意权重（含负），求 SSSP。若存在从 s 可达的负权环，返回 −∞ 或返回该环。

12.2 两个关键引理

- **引理 1**：若无负权环，则有限的最短路可以取**简单路径**（不重复顶点）。证明：若最短路含环，环权非负，去掉环不会变长。于是**简单路径至多 |V|−1 条边**。
- 定义 **k−边距离 δ_k(s,v)**：使用至多 k 条边的路径的最小权重。
- **引理 2**：令 δ(s,v) 有限，则 δ(s,v) = δ_{|V|−1}(s,v)；若 δ_{|V|}(s,v) < δ_{|V|−1}(s,v)，则 v 是**负环见证（witness）**，δ(s,v) = −∞。且每个 δ = −∞ 的顶点都从某个见证可达。

12.3 分层图构造（讲义的讲法）

把图「复制」|V|+1 层，顶点 v_k 表示「用至多 k 条边到达 v」：

- 层间零权边 (v_{k−1}, v_k)（表示「不用第 k 条边」）；
- 每条原边 (u,v) 生成 (u_{k−1}, v_k)，权重 w(u,v)。

新图 G′ 是 DAG，跑 DAG 松弛即得所有 δ_k。G′ 有 |V|(|V|+1) 个顶点、|V|(|V|+|E|) 条边，故总时间 **O(|V|·|E|)**。

```
def bellman_ford(adj, s, V):
    d = {v: float("inf") for v in V}
    d[s] = 0
    for _ in range(len(V) - 1):            # |V|-1 轮
        changed = False
        for u in V:
            if d[u] == float("inf"): continue
            for v, w in adj[u]:
                if d[v] > d[u] + w:
                    d[v] = d[u] + w
                    changed = True
        if not changed: break              # 提前退出：已经收敛
    # 第 |V| 轮还能松弛 => 存在从 s 可达的负环
    for u in V:
        if d[u] == float("inf"): continue
        for v, w in adj[u]:
            if d[v] > d[u] + w:
                raise ValueError("negative cycle reachable from s")
    return d
```

上面的写法是**传统的单数组 in-place 版**（每轮遍历所有边）。讲义明确指出：它与分层图版的渐近代价相同，但估计值不再对应「恰好 k 条边」，**正确性分析更微妙**；分层图版则能顺便返回负环。

12.4 复杂度

版本	时间	空间	能否返回负环
分层图 + DAG 松弛	$O(\|V\|\cdot\|E\|)$	$O(\|V\|^2)$	能（沿 $\|V\|+1$ 个顶点的路径找环）
传统 $\|V\|$ 轮 in-place	$O(\|V\|\cdot\|E\|)$	$O(\|V\|)$	只能报告存在
剪枝到 s 的可达子图	$O(\|V\|\cdot\|E\|)$，常更快	$O(\|V\|)$	同传统

12.5 实现陷阱

- **陷阱 12-1**：轮数写成 $|V|$ 而不是 $|V|-1$，或写成 $|E|$。正确的是 $|V|-1$ 轮 + 1 轮检测。
- **陷阱 12-2**：负环检测放在循环内部（每轮都查）会误报——必须等 $|V|-1$ 轮全部收敛后再查。
- **陷阱 12-3**：无向图里只要有一条负权边，就等价于存在负环（来回走两次），所以讲义第 12 章开头用一道热身题把讨论限制在有向图。
- **陷阱 12-4**：提前退出（`changed == False`）是重要的常数优化，最短路在稀疏图上常常几轮就收敛。
- **陷阱 12-5**：只报告「存在负环」不等于给出所有 $\delta = -\infty$ 的点。要从见证出发再做一次可达性搜索。
- **陷阱 12-6**：Dijkstra 的代码套上「跑 $|V|$ 轮」并不会变成 Bellman-Ford——Bellman-Ford 的关键是**每轮松弛所有边**，与优先队列无关。

第 13 章 Dijkstra 算法（L13）

13.1 问题

一般图 + **非负权重**，求 SSSP，要求比 Bellman-Ford 快。

13.2 关键观察

- **观察 1**：非负权下，沿最短路距离单调不减：若 u 在 $s \to v$ 的最短路上，则 $\delta(s,u) \leqslant \delta(s,v)$。于是「距离 $\leqslant x$ 的可达集 V_x」可以一个一个顶点地长大。
- **观察 2**：若已知顶点按距离递增的顺序，就可以删掉逆序边，剩下的图做 DAG 松弛即可。

Dijkstra 就是把「按距离递增出队」用优先队列自动化。

13.3 需要的接口：可变优先队列

```
Q.build(X)              初始化
Q.delete_min()          取出最小 key 的元素
Q.decrease_key(id, k)   把指定 id 的 key 降到 k
```

实现方式：交叉链接一个普通优先队列 Q′ 和一个 **字典 D：id → Q′ 中的位置**。顶点 id 取 0…|V|−1，D 用直接寻址数组。

```
DIJKSTRA(G, s)
1  for v in V: d[v] = INF
2  d[s] = 0
3  Q.build([(v, d[v]) for v in V])
4  while Q not empty:
5      (u, du) = Q.delete_min()
6      for (v, w) in Adj(u):
7          if d[v] > du + w:
8              d[v] = du + w
9              Q.decrease_key(v, d[v])
```

13.4 正确性（一句话版）

对出队顺序归纳：设 v′ 是第 k 个出队的顶点，取一条最短路 π，令 (x,y) 是 π 上第一条「y 尚未出队」的边。x 出队时松弛过 (x,y)，故 $d(y) \leqslant \delta(s,y) \leqslant \delta(s,v') \leqslant d(v')$；又 v′ 是队列中最小，故 $d(v') \leqslant d(y)$，全链相等。**非负性用在 $\delta(s,y) \leqslant \delta(s,v')$ 这一步**（y 在 v′ 的最短路上）。完整证明见讲义 L13。

13.5 复杂度（讲义原文表）

Q′ 的实现	build	delete_min	decrease_key	Dijkstra 总时间（n=|V|=O(|E|)）
数组	n	n	1	$O(\|V\|^2)$
二叉堆	n	log n（摊还）	log n	$O(\|E\| \log \|V\|)$
Fibonacci 堆	n	log n（摊还）	1（摊还）	$O(\|E\| + \|V\| \log \|V\|)$

选型（讲义结论）：

- **稠密图** $|E| = \Theta(|V|^2)$：数组版 $O(|V|^2)$ 最快、常数最小；
- **稀疏图** $|E| = \Theta(|V|)$：二叉堆版 $O(|V| \log |V|)$；
- Fibonacci 堆理论上全能，但实践几乎不用（6.006 不要求，CLRS 有专章）；
- 理论题里默认 Dijkstra 是 $O(|E| + |V| \log |V|)$。

13.6 Python 实现：懒删除（lazy deletion）

标准库 `heapq` 没有 `decrease_key`，工程上的标准做法是**每次松弛都 push 一个新的 (dist, v) 条目**，出队时若 `dist` 与当前 `d[v]` 不符就丢弃（stale entry）。

```
def dijkstra(adj, s, V):            # heapq 没有 decrease_key => 懒删除
    d = {v: float("inf") for v in V}; d[s] = 0
    pq, done = [(0, s)], set()
    while pq:
        du, u = heapq.heappop(pq)
        if u in done: continue                    # 过期条目，丢弃
        done.add(u)
        for v, w in adj[u]:
            if d[v] > du + w:
                d[v] = du + w; heapq.heappush(pq, (d[v], v))
    return d
```

- 时间 O((|V| + |E|) log |V|)：每条边最多触发一次 push，堆里至多 |E| 条。
- 空间 O(|V| + |E|)（堆里可能有重复条目）。

13.7 实现陷阱

- **陷阱 13–1**：把 Dijkstra 用在负权图上会得到错误结果（且不报错）。判断依据：只要有一条负权边，就必须换 Bellman–Ford 或 Johnson。
- **陷阱 13–2**：懒删除版本忘了 `done` 集合，同一个顶点会被处理多次，复杂度退化。
- **陷阱 13–3**：把 `if d[v] > du + w` 改成 `>=` 会在零权边上反复入堆。
- **陷阱 13–4**：`done` 里的点再次出队时直接 `continue`，但**不要**再更新 d——它已经定稿。
- **陷阱 13–5**：稠密图上用 heapq 反而比 O(|V|²) 的数组扫描慢（log 因子 + 堆操作开销）。|V| = 1000、|E| ≈ 10^6 时值得手写数组版。
- **陷阱 13–6**：要输出路径时在松弛成功时记 `P[v] = u`，而不是出队时记（出队时 u 可能已被后来的更短路径覆盖）。

第 14 章 全源最短路与 Johnson 算法（L14）

14.1 问题

APSP：输出所有 δ(u,v)，若存在负权环则中止。直接跑 |V| 次 SSSP 其实「还不错」，因为输出本身就有 |V|² 个值：

图/权重	|V| 次 SSSP 的总时间
无权且权重有界	|V|·O(|V|+|E|)
DAG	|V|·O(|V|+|E|)
非负	|V|·O(|V| log|V| + |E|)
任意	|V|·O(|V|·|E|)

目标：任意权重也能做到 |V|·O(|V| log|V| + |E|)。

14.2 重赋权（reweighting）

天真的做法「给每条边加同一个常数」是**错的**：它偏向边数少的路径。

正确做法：给每个顶点一个**势（potential）** h(v)，令

$$w'(u,v) = w(u,v) + h(u) - h(v)$$

则对任意路径 π = (v_0,…,v_k)，w′(π) = w(π) + h(v_0) − h(v_k)——中间的 h 全部望远镜式抵消。**所有同端点的路径被平移了相同的量**，所以最短路不变。

要让 w′ ⩾ 0，需要 w(u,v) + h(u) − h(v) ⩾ 0，即 h(v) ⩽ h(u) + w(u,v)——这正是三角不等式。于是取 h(v) = δ(x,v)（某个源 x 到 v 的距离）即可。

但图可能不连通，不存在能到达所有点的 x。**解法：加一个超级源 x，向每个顶点连一条 0 权边**。于是 δ(x,v) ⩽ 0，且 x 没有入边、不会引入新环。

14.3 算法

```
JOHNSON(G, w)
1  构造 Gx：加顶点 x，对每个 v ∈ V 加边 (x, v, 0)
2  (d, ok) = BELLMAN_FORD(Gx, x)        # O(|V||E|)
3  if not ok: abort                     # 有负环
4  对每条边 (u,v)：w'(u,v) = w(u,v) + d[u] − d[v]     # 非负
5  for u in V: 对 G' 跑 DIJKSTRA(u)                  # |V| · O(|V|log|V| + |E|)
6  δ(u,v) = δ'(u,v) − d[u] + d[v]                     # 还原
```

```
def johnson(adj, V):
    adj2 = {**{v: list(adj[v]) for v in V}, "x": [(v, 0) for v in V]}
    h = bellman_ford(adj2, "x", list(V) + ["x"])          # 有负环会抛异常
    w2 = {u: [(v, w + h[u] − h[v]) for (v, w) in adj[u]] for u in V}
    return {u: {v: (d2[v] − h[u] + h[v] if d2[v] < float("inf") else float("inf"))
                for v in (d2 := dijkstra(w2, u, V))} for u in V}
```

14.4 复杂度

步骤	时间
构造 Gx	$O(\|V\| + \|E\|)$
Bellman–Ford（检测负环 + 求势）	$O(\|V\| \cdot \|E\|)$
构造 G′	$O(\|V\| + \|E\|)$
|V| 次 Dijkstra	$O(\|V\| \cdot (\|V\| \log\|V\| + \|E\|))$
还原距离	$O(\|V\|^2)$
合计	**$O(\|V\|^2 \log\|V\| + \|V\| \cdot \|E\|)$**

意义：**把「带负权的 APSP」归约成「非负权的 APSP」**。同一件事的另一条路是第 17 章的 Floyd–Warshall，$O(|V|^3)$，稠密图上与 Johnson 同阶，但实现简单得多。

14.5 实现陷阱

- **陷阱 14–1**：忘了先跑 Bellman–Ford 检测负环，直接算势。有负环时 $\delta(x,v) = -\infty$，重赋权无意义。
- **陷阱 14–2**：还原距离时符号搞反。记住 $w'(\pi) = w(\pi) + h(u) - h(v)$（起点减终点），所以 $\delta = \delta' - h(u) + h(v)$。
- **陷阱 14–3**：只对「可达对」还原。不可达仍是 ∞，不要拿 ∞ 去减势（会得到 nan 或巨大负数）。
- **陷阱 14–4**：误以为重赋权后原图的最短路「长度」不变。变的是每条路径的绝对权重（平移 h(起点) −h(终点)），不变的是「哪条路径最短」。
- **陷阱 14–5**：把超级源接到每个顶点时用了非零权重，导致势不满足三角不等式。

第 15 章 递归算法与动态规划基础（L15）

15.1 问题

前半门课是「归约到已知的数据结构或算法」；后半门课是「自己设计递归算法」。递归算法有一个统一的视角：把**函数调用看成有向图**——函数是顶点，A 调用 B 就是边 A→B。若能终止，这个**依赖图必须无环**。按图形状给算法分类（讲义原文）：

类别	依赖图形状	例子
Brute Force	星形（一个根连所有）	排列枚举
Decrease & Conquer	链	插入排序、选择排序

类别	依赖图形状	例子
Divide & Conquer	树	归并排序、Karatsuba
Dynamic Programming	**DAG（入度可 > 1）**	Fibonacci、最短路、LCS
Greedy / Incremental	子图	区间调度

DP 的本质就一句话：**子问题依赖有重叠（DAG 而不是树）时，把子问题的解存下来复用**。

15.2 SRT BOT：六步框架（本课最重要的模板）

讲义把它写成递归解题的固定流程，Quiz 3 与 Final 的 DP 题按这六步给分：

1. **S**ubproblem（子问题定义）：用参数说明子问题含义。常见形式是前缀、后缀、连续子串，或「加一个辅助状态」。
2. **R**elate（递推关系）：找出一个「若知道答案就能把问题化小」的问题，然后**局部暴力枚举**它的所有答案。
3. **T**opological order（拓扑序）：论证依赖无环，并给出计算顺序。
4. **B**ase cases（基例）：递推失效处的直接答案。
5. **O**riginal problem（原问题）：如何由子问题得到答案（可能需要父指针复原方案）。
6. **T**ime（时间分析）：Σ_{x∈X} work(x)；若每个子问题 work 都是 O(W)，就是 |X|·O(W)。

15.3 Fibonacci：从指数到线性

```
F(i) = F(i-1) + F(i-2),  F(0)=0, F(1)=1
```

朴素递归 T(n) = T(n−1) + T(n−2) + O(1) > 2T(n−2) ⇒ Ω(2^{n/2})，因为 F(k) 被重复计算 F(n−k) 次。

n+1 个子问题、< 2n 条依赖边 ⇒ 记忆化后 O(n) 次加法。

```
def fib_memo(n):                    # top-down：递归 + 记忆化
    memo = {0: 0, 1: 1}
    def F(i):
        if i not in memo: memo[i] = F(i-1) + F(i-2)
        return memo[i]
    return F(n)

def fib_iter(n):                    # bottom-up：按拓扑序迭代
    a, b = 0, 1
    for _ in range(n): a, b = b, a + b
    return a
```

> **注意数值代价**：讲义提醒 F_n 有 Θ(n) 比特，加法不是 O(1) 而是 O(⌈n/w⌉)，所以总时间是 O(n + n^2/w)，不是纯 O(n)。这是「Word-RAM 假设被打破」的经典例子。

15.4 DAG 最短路就是 DP

- 子问题：δ(s,v)，v ∈ V；
- 递推：δ(s,v) = min{δ(s,u) + w(u,v) | u ∈ Adj^-(v)} ∪ {∞}；
- 拓扑序：G 的拓扑序；
- 基例：δ(s,s) = 0；
- 原问题：全部子问题；
- 时间：Σ_v O(1 + |Adj^-(v)|) = O(|V|+|E|)。

DAG 松弛（第 11 章）与这个 DP **算的是同一组值**，只是前者从 u 出发看 Adj^+(u)、边算边取 min，后者从 v 出发看 Adj^-(v)。

15.5 Bowling：一维 DP 的样板

问题：n 个球瓶，瓶 i 价值 v_i。一球可击倒 1 个瓶（得 v_i）或相邻 2 个瓶（得 v_i·v_{i+1}），每瓶最多被击倒一次，求最大得分。

分治版（区间 DP）会得到 O(n^2) 且难以推广；**后缀 DP** 才是正解：

- 子问题：B(i) = 只考虑瓶 i…n−1 时的最大得分；
- 递推：对第一个瓶做局部暴力——跳过、单独击倒、与下一个一起击倒： B(i) = max{B(i+1), v_i + B(i+1), v_i·v_{i+1} + B(i+2)}；
- 拓扑序：i 递减；
- 基例：B(n) = B(n+1) = 0；
- 原问题：B(0)；
- 时间：Θ(n) 个子问题 × Θ(1) = **Θ(n)**。

```
def bowl(v):
    n = len(v); B = [0] * (n + 2)                # B[n] = B[n+1] = 0
    for i in reversed(range(n)):                 # 拓扑序：i 递减
        cand = [B[i+1], v[i] + B[i+1]]           # 跳过 / 单独击倒
        if i + 1 < n: cand.append(v[i] * v[i+1] + B[i+2])   # 两瓶一起
        B[i] = max(cand)
    return B[0]
```

15.6 复杂度与陷阱

- 记忆化（top-down）：只算可达子问题，写法自然，但有递归深度与字典开销。
- 自底向上（bottom-up）：无递归、常数小，但会算所有子问题（包括不可达的）。
- **陷阱 15-1**：memo 的 key 不完整（漏了某个影响状态参数），导致不同子问题被当成同一个。
- **陷阱 15-2**：拓扑序写反（该递减的写成递增），读到未初始化的值。Python 里表现为「结果偏小」而不是报错。
- **陷阱 15-3**：基例只写一个边界。Bowling 需要 B(n) 与 B(n+1) 两个，少一个就索引越界。
- **陷阱 15-4**：忘记「可以跳过」这一分支（B(i+1)），把「必须击倒」当成约束。
- **陷阱 15-5**：在递归函数里用可变默认参数（`memo={}`）当缓存，跨调用污染。用闭包或显式传参。

第 16 章 DP 的子问题设计：LCS、LIS、取硬币（L16）

16.1 问题

DP 的难点不在写递推，而在**设计子问题**。本章三个例子分别演示三种设计手法：二维后缀、加约束、加状态。

16.2 最长公共子序列（LCS）

给定两个串 A、B，求最长的、同时是两者子序列（不要求连续）的串。例：A = hieroglyphology、B = michaelangelo，答案长度 5（hello / heglo / iello / ieglo）。

- 子问题：x(i,j) = A[i:] 与 B[j:] 的 LCS 长度，$0 \leqslant i \leqslant |A|$、$0 \leqslant j \leqslant |B|$；
- 递推：若 A[i] == B[j]，则 x(i,j) = x(i+1,j+1) + 1；否则 x(i,j) = max{x(i+1,j), x(i,j+1)}；
- 拓扑序：i+j 递减（代码里写「i 递减、j 递减」也行）；
- 基例：x(i,|B|) = x(|A|,j) = 0；
- 原问题：x(0,0)；要复原串就存父指针，父指针同时让 i、j 都前进时输出该字符；
- 时间：$\Theta(|A|\cdot|B|)$ 个子问题 × O(1) = **$O(|A|\cdot|B|)$**。

```
def lcs(A, B):
    a, b = len(A), len(B)
    x = [[0] * (b + 1) for _ in range(a + 1)]
    for i in reversed(range(a)):
        for j in reversed(range(b)):
            x[i][j] = x[i+1][j+1] + 1 if A[i] == B[j] else max(x[i+1][j], x[i][j+1])
    return x[0][0]
```

16.3 最长递增子序列（LIS）与「加约束」

A = carbohydrate ⇒ LIS 是 abort（长度 5）。

错误的起点：x(i) = A[i:] 的 LIS 长度。这样递推时无法保证「递增」——你不知道下一个选中的元素要和谁比。

修正：给子问题**加约束**——x(i) = 「必须以 A[i] 开头的」A[i:] 的 LIS 长度。

- 递推：x(i) = max{1 + x(j) | i < j < |A|, A[j] > A[i]} ∪ {1}；
- 拓扑序：i 递减；
- 基例：不需要（「A[i] 是最后一个」由 ∪{1} 覆盖）；
- 原问题：max{x(i)}（第一个元素要枚举）；
- 时间：O(|A|²)。

```
def lis(A):
    x = [1] * len(A)
    for i in reversed(range(len(A))):
        for j in range(i + 1, len(A)):
            if A[j] > A[i]: x[i] = max(x[i], 1 + x[j])
    return max(x) if x else 0
```

> 讲义留了一道练习：用 AVL 增广把每个子问题的 O(n) 工作量降到 O(log n)，得到 O(n log n) 的 LIS。思路是对「值」建 AVL，结点存「以该值为起点的 LIS 长度」的子树最大值——这正是第 7 章增广技巧与第 16 章 DP 的交汇点。

16.4 交替取硬币（zero-sum 博弈）

n 枚硬币 $v_0 \ldots v_{n-1}$，两人轮流从**剩余序列的两端**取一枚，我先手，求我能拿到的最大总值。

解法一（利用零和）：

- 子问题：x(i,j) = 从硬币 i…j 开始我能拿到的最大值；
- 递推：我必取 i 或 j；取完后轮到对方，对方能拿到 x(i+1,j) 或 x(i,j−1)，所以 x(i,j) = max{ v_i + (前缀和 − v_i − x(i+1,j)), v_j + (前缀和 − v_j − x(i,j−1)) }；

- 每步要算区间和 ⇒ 每个子问题 $\Theta(n)$ ⇒ 总 **$\Theta(n^3)$**；讲义留练习：前缀和预处理后降到 $\Theta(n^2)$。

解法二（扩张状态，不依赖零和）：

- 子问题：x(i,j,p) = 「接下来轮到 p 走」时我能拿到的最大值，p ∈ {me, you}；
- x(i,j,me) = max{v_i + x(i+1,j,you), v_j + x(i,j−1,you)}；
- x(i,j,you) = min{x(i+1,j,me), x(i,j−1,me)}（对方会让我拿得最少）；
- 每个子问题 $\Theta(1)$ ⇒ **$\Theta(n^2)$**。

解法二更慢吗？不，它更快（n^2 vs n^3），而且**可推广**到非零和。这就是「扩张子问题」的价值：多一维状态换来更紧的递推。

16.5 复杂度与陷阱

问题	子问题数	每个的 work	时间
LCS	(|A|+1)(|B|+1)	O(1)	O(|A||B|)
LIS	|A|	O(|A|)	O(|A|²)（可优化到 O(n log n)）
取硬币（零和版）	$\Theta(n^2)$	$\Theta(n)$	$\Theta(n^3)$（可优化到 $\Theta(n^2)$）
取硬币（状态扩张版）	$\Theta(n^2)$	$\Theta(1)$	$\Theta(n^2)$

- **陷阱 16–1**：LCS 里把「子序列」误读成「子串」（连续）。子串版是另一道题，递推完全不同。
- **陷阱 16–2**：LIS 用「前缀」而不是「后缀 + 必须包含 A[i]」的约束，递推写不出来。
- **陷阱 16–3**：取硬币的「零和」推导只对两人都最优时成立。若题目允许「对方随机走」，就不能用减法。
- **陷阱 16–4**：二维 DP 的循环方向要与拓扑序一致；LCS 里 i、j 都递减是对的，i 递减 j 递增就错了。
- **陷阱 16–5**：复原方案时只存了「长度」没存「选择」，最后只能输出数值不能输出方案。

第 17 章 DP 进阶：SSSP、Floyd–Warshall、括号化、指法（L17）

17.1 把 Bellman–Ford 看成 DP（状态扩张 Θ(n) 倍）

- 子问题：δ_k(s,v) = 用**至多 k 条边**的最短路径权重，v ∈ V、0 ⩽ k ⩽ |V|；
- 递推：猜最后一条边——δ_k(s,v) = min{δ_{k−1}(s,u) + w(u,v)} ∪ {δ_{k−1}(s,v)}；
- 拓扑序：k 递增；
- 基例：δ_0(s,s) = 0，其余 ∞；
- 原问题：若最短路存在则 δ(s,v) = δ_{|V|−1}(s,v)；若 δ_{|V|}(s,v) < δ_{|V|−1}(s,v) 则有负环；

- 时间：|V|·(1+|V|) 个子问题，每个 O(deg⁻(v))，合计 **O(|V|·|E|)**。

这就是 Bellman–Ford，只是按 k 而不是按边遍历。**「把环状依赖用一维状态展开成 DAG」是本课最通用的建模手法。**

17.2 Floyd—Warshall：常数分支的 APSP

- 子问题：x(u,v,k) = 只允许经过顶点集 {1,…,k}（外加 u、v 本身）的 u→v 最小权重；
- 递推：x(u,v,k) = min{ x(u,k,k−1) + x(k,v,k−1), x(u,v,k−1) }；
- 拓扑序：k 递增；
- 基例：x(u,u,0) = 0；x(u,v,0) = w(u,v)（有边）；否则 ∞；
- 原问题：x(u,v,|V|)；
- 时间：**O(|V|³)**。

关键洞察：猜「中间经过哪个顶点」有 |V| 种选择，但把「允许的中间顶点集合」当作状态后，每个子问题只有**两个**依赖（用 k 或不用 k）。分支因子从 O(|E|) 降到 O(1)，正是 O(|V|³) 的来源。

```
def floyd_warshall(V, w):
    INF, n = float("inf"), len(V)
    d = [[0 if i == j else INF for j in range(n)] for i in range(n)]
    for (u, v), ww in w.items(): d[u][v] = min(d[u][v], ww)
    for k in range(n):                              # k 必须最外层
        for i in range(n):
            for j in range(n):
                if d[i][k] + d[k][j] < d[i][j]: d[i][j] = d[i][k] + d[k][j]
    return d                                        # d[i][i] < 0 即负环
```

17.3 算术括号化（子串 DP + min/max 双状态）

给 $a_0 *_1 a_1 *_2 \dots *_{n-1} a_n$（$* \in \{+, \times\}$，a_i 可为负），求最大化表达式值的加括号方式。例：7 + 4 × 3 + 5 ⇒ ((7)+(4))×((3)+(5)) = 88。

- 为什么不能只存最大值？因为负数相乘：两个「最小」（最负）相乘可能变成最大。所以子问题要**扩张成 min 与 max 两套**： x(i,j,opt) = 把 a_i…a_j 括号化能得到的最小/最大值；
- 递推：猜最外层最后做的那次运算位置 k， x(i,j,opt) = opt over k, opt′, opt″ of (x(i,k,opt′) $*_k$ x(k,j,opt″))；
- 拓扑序：区间长度 j−i 递增；
- 基例：x(i,i+1,opt) = a_i；
- 时间：O(n²) 个子问题 × O(n)·2·2 的 work = **O(n³)**。

17.4 钢琴指法（状态 = 局部决策）

给 n 个音符 $t_0 \ldots t_{n-1}$，F 根手指，给定换指难度 d(t,f,t′,f′)，求总难度最小的指法。

错误起点：x(i) = 弹 i…n−1 的最小难度。递推时需要知道「i+1 用哪根手指」，而不同的起始手指会同时影响 x(i+1) 与 d(·)，信息不够。

修正：给子问题加一个「起始手指」状态——x(i,f) = 从音符 i 开始、用 f 指弹 t_i 的最小总难度。递推枚举下一根手指 f′： x(i,f) = min{ x(i+1,f′) + d(t_i,f,t_{i+1},f′) }。

- 时间：Θ(nF) 子问题 × Θ(F) = **$\Theta(nF^2)$**；F=5 时就是 Θ(n)。
- 推广：吉他加「弦」维度，F → F·S，时间 $\Theta(nF^2S^2)$；同时弹多个音时，指法是一个映射，状态数 T^F，时间 Θ(nT^{2F})。

17.5 复杂度与陷阱

- **陷阱 17-1**：Floyd–Warshall 的三重循环把 k 写在最内层。k 必须最外层，否则用的是「本轮已更新」的中间值，结果错。
- **陷阱 17-2**：FW 的原地更新是允许的（因为 d[i][k]、d[k][j] 在第 k 轮不会变），但不要因此以为循环顺序可以随便换。
- **陷阱 17-3**：括号化只存 max，遇到负数乘法就错。必须同时存 min。
- **陷阱 17-4**：指法题忘了「同一根手指连续弹同一个音」的惩罚（legato 要求 t ≠ t′，否则无穷大），导致退化解。
- **陷阱 17-5**：Bellman–Ford 的 DP 版把「至多 k 条边」写成「恰好 k 条边」，基例与终止条件都要改，否则负环判定失效。

第 18 章 伪多项式时间：切杆与子集和（L18）

18.1 问题：什么叫「多项式」

讲义的定义：**（强）多项式时间**指运行时间被「输入规模（以字为单位）」的常数次多项式界住。注意「输入规模」不是「最大的那个数」，而是「有多少个字」。

18.2 切杆（Rod Cutting）

长 L 的杆，长度 ℓ 的价值 v(ℓ)，切成若干段使总价值最大。反例（讲义原文）：L=7、v = [0,1,10,13,18,20,31,32]，贪心取「单位价值最高」的 ℓ=6 得 32，但最优是 v[2]+v[2]+v[3] = 33。

- 子问题：x(ℓ) = 长度 ℓ 的最大价值；

- 递推：$x(\ell) = \max\{ v(p) + x(\ell-p) \mid 1 \leqslant p \leqslant \ell \}$；
- 拓扑序：ℓ 递增；
- 基例：$x(0) = 0$；
- 时间：L+1 个子问题 × O(L) = **$O(L^2)$**。

输入规模是 L+1 个字（一个 L 加 L 个价值），$O(L^2)$ 是它的多项式 ⇒ **这是真多项式算法**。

```
def cut_rod(L, v):
    x, parent = [0] * (L + 1), [None] * (L + 1)
    for l in range(1, L + 1):
        for p in range(1, l + 1):
            if v[p] + x[l-p] > x[l]: x[l], parent[l] = v[p] + x[l-p], l - p
    return x[L]                                  # parent 链可复原切法
```

18.3 子集和（Subset Sum）

给 n 个正整数 A 与目标 T，问是否存在子集和恰为 T。例：A = (1,3,4,12,19,21,22)、T=47，A′ = {3,4,19,21} 成立。

- 子问题：x(i,t) = A[i:] 能否凑出 t，$0 \leqslant i \leqslant n$、$0 \leqslant t \leqslant T$；
- 递推：猜 a_i 是否被选——x(i,t) = x(i+1, t−A[i])（若 $t \geqslant A[i]$）**或** x(i+1, t)；
- 拓扑序：i 递减；
- 基例：x(i,0) = YES；x(n,t>0) = NO；
- 时间：O(nT) 个子问题 × O(1) = **O(nT)**。

```
def subset_sum(A, T):
    n = len(A)
    x = [[False] * (T + 1) for _ in range(n + 1)]
    for i in range(n + 1): x[i][0] = True        # 基例：凑出 0
    for i in reversed(range(n)):
        for t in range(T + 1):
            x[i][t] = x[i+1][t] or (t >= A[i] and x[i+1][t - A[i]])
    return x[0][T]                               # O(nT)，伪多项式
```

它是不是多项式？ 输入规模是 n+1 个字，但 T 可以大到 2^w，而 w 没有上界（w = n 也不离谱），此时 O(nT) = O(n·2^n) 是指数级。所以它是**伪多项式（pseudopolynomial）**。

18.4 三个层次的复杂度

概念	定义	例子
强多项式	输入规模（字数）的多项式	切杆 $O(L^2)$（因为同时给了 L 个价值）

概念	定义	例子
弱多项式	输入规模按**比特**计的多项式	基数排序 O(n log_n u)
伪多项式	输入规模**和输入整数大小**的多项式	子集和 O(nT)、计数排序 O(n+u)、Fibonacci O(n)

子集和是**弱 NP 完全**（有伪多项式算法，但没有多项式算法除非 P=NP）；3-Partition 是**强 NP 完全**（连伪多项式算法都没有，除非 P=NP）。

18.5 陷阱

- **陷阱 18-1**：把 O(nT) 说成多项式。考试会扣分，必须点名「伪多项式」。
- **陷阱 18-2**：T 很大时二维表爆内存。可用滚动数组把空间降到 O(T)，或用 Python 大整数做 bitset：`bits = 1; for a in A: bits |= bits << a`，最后看第 T 位是否为 1（工程技巧，位并行，复杂度 O(nT/64)）。
- **陷阱 18-3**：top-down 记忆化只访问可达状态（讲义例子：35 个子问题只访问 14 个），大数据下比 bottom-up 省。
- **陷阱 18-4**：`x(i, t)` 的 t 维度从 0 到 T，边界 t=0 必须显式置 True。
- **陷阱 18-5**：切杆的「第一次切 p」与「最后一次切 p」是两种等价建模，但 parent 指针的含义不同，复原时要对应。

第 19 章 复杂度：P、NP 与归约（L19）

19.1 判定问题

判定问题（decision problem）把输入映射到 YES/NO。把优化问题改写成判定问题是 NP 理论的入口：

问题	判定版本
s-t 最短路	是否存在权重 ⩽ d 的 s→t 路径?
负环	是否存在负权环?
最长简单路径	是否存在权重 ⩾ d 的简单路径?
子集和	是否存在和恰为 S 的子集?
Tetris	给定方块序列与棋盘，能否存活?
Chess	给定棋盘，先手能否强制获胜?
Halting	给定程序与输入，是否终止?

可判定（decidable）：存在有限时间求解的程序。由于程序是可数无穷、问题是不可数无穷（Cantor 对角线），**大多数问题不可判定**；停机问题是典型。

19.2 三个类

类	含义	备注
R	有限时间可判定	recursive languages
EXP	2^{n^{O(1)}} 时间可判定	我们想到的多数问题在这里
P	n^{O(1)} 时间可判定	本课的主战场

$P \subsetneq EXP \subsetneq R$（时间层次定理，6.045 的内容）。国际象棋是 $EXP \setminus P$ 的例子。

19.3 NP：用证书定义

NP 是「存在多项式时间的**验证器** V」的判定问题集合：V 接收输入 I 与长度 polynomial(|I|) 的证书 c，满足

- V 对 |I| 是多项式时间；
- 若 I 是 YES，存在某个 c 使 V(I,c) = YES；
- 若 I 是 NO，对任意 c 都有 V(I,c) = NO。

问题	证书	验证器做什么
s–t 最短路	一条路径 P	把 P 的权重加起来看是否 $\leqslant d$
负环	一个环 C	把 C 的权重加起来看是否 < 0
最长简单路径	一条路径 P	检查 P 简单且权重 $\geqslant d$
子集和	一个子集 A'	检查 $A' \subseteq A$ 且和为 S
Tetris	一串操作	检查这串操作能存活

- $P \subseteq NP$：验证器直接求解，忽略证书。
- $NP \subseteq EXP$：枚举所有 $\leqslant 2^{n^{O(1)}}$ 个证书，逐个跑验证器。
- **P = NP?** 未解，克雷研究所百万美元问题。主流看法是 $P \subsetneq NP$。

19.4 归约与 NP 完全

- **归约** $A \to B$：把 A 的实例转成 B 的实例，用 B 的算法解 A。于是 B 至少和 A 一样难（$A \leqslant B$）。
- **NP-hard**：NP 中每个问题都能多项式归约到它。
- **NP-complete** = NP ∩ NP-hard。所有 NP 完全问题互相归约。

- 第一个 NP 完全问题是 **Circuit SAT**（每个判定问题都能归约到「布尔电路可满足」）。

讲义列的 NP 完全问题（部分）：子集和（弱）、3-Partition（强）、矩形装箱、拼图、n 个串的 LCS、最长简单路径、TSP、3D 避障最短路、3-着色（2-着色在 P）、最大团、SAT、扫雷/数独、多数电子游戏。

19.5 算法设计的三个归约套路（讲义原文表）

原问题	转换	目标问题
无权最短路	所有边权设为 1	加权最短路
整数权最短路	把边细分	无权最短路
最长路径	权重取负	最短路

19.6 陷阱

- **陷阱 19-1**：归约方向搞反。要证「A 是 NP-hard」，是把**已知的 NP-hard 问题归约到 A**（A 接收），不是把 A 归约出去。
- **陷阱 19-2**：把「NP-hard」等同于「不可解」。NP-hard 问题在**特定结构**下常有高效算法（比如最长路径在 DAG 上是线性的）。
- **陷阱 19-3**：把「伪多项式」当「多项式」。见第 18 章。
- **陷阱 19-4**：证书长度必须多项式。允许指数长证书会把 NP 撑大。
- **陷阱 19-5**：说「这个问题在 EXP 所以不在 P」。EXP \ P 的存在性由层次定理保证，但单个问题要具体论证。

第 20 章 数据结构专题：六种容器的横向对比

把第 2、4、6、7、8 章的结论并到一张表里（n = 元素数，u = key 空间，h = 树高，(e) 期望，(a) 摊还）：

结构	find	insert	delete	min/max	前驱/后继	按下标	空间	有序
静态数组	n	—	—	n	n	**1**	n	否
动态数组	n	1(a)（尾部）	1(a)（尾部）	n	n	**1**	$\leqslant 2n$	否
单/双向链表	n	1（已知位置）	1（已知位置）	n	n	n	n	否
有序数组	**log n**	n	n	**1**	log n	1	n	是

结构	find	insert	delete	min/max	前驱/后继	按下标	空间	有序
哈希表（链地址）	**1(e)**	**1(a)(e)**	**1(a)(e)**	n	n	—	Θ(n)	否
直接寻址数组	**1**	**1**	**1**	u	u	1	**u**	是（按 key）
BST（不平衡）	h	h	h	h	h	h	n	是
AVL	**log n**	**log n**	**log n**	log n	log n	log n	n	是
二叉堆	n	**log n**	log n（只能删极值）	**1**	n	—	n	否（局部序）

20.1 选型决策（工程版）

选型口诀：**要顺序 → AVL；不要顺序 → 哈希；只要极值 → 堆；按下标 → 数组；两端增删 → deque；已知结点删除 → 双向链表 + 哈希（LRU）**。

20.2 动态数组的三个数字

- 扩容因子 2（或 1.5）：摊还 O(1)，空间 ⩽ 2n（1.5 时 ⩽ 1.5n 但搬运更频繁）。
- 缩容阈值必须是**扩容阈值的一半以下**（讲义：r_d = 1/4、r_i = 1/2），否则抖动。
- 任意 ε > 0 可把额外空间压到 (1+ε)n。

20.3 哈希表的三个不变量

1. 负载因子 α = n/m = O(1)（讲义取 m = Θ(n)）；
2. 哈希函数从**全域族**随机取，且每次重建后重新取；
3. 扩容/缩容时**重新哈希所有元素**（不能直接搬，因为 m 变了）。

20.4 AVL 的四个数字

- 平衡条件：|skew| ⩽ 1；
- 高度界：$h \leqslant 2\log_2 n$（由 F(h) ⩾ 2^{h/2} 推出）；
- 插入旋转次数 ⩽ 1；删除旋转次数 O(log n)；
- 增广字段：height、size（或子树 max/min 指针）。

20.5 堆的两个公式与一个界

- `left = 2i+1`、`right = 2i+2`、`parent = (i-1)//2`（0-index）；
- 建堆 Θ(n)（自底向上），单次 insert/delete O(log n)；
- 高度 ⌈lg n⌉，且堆**不是** BST，中序无序。

20.6 并查集（Union-Find）：6.006 之外的必备结构

> **诚实声明**：并查集**不在 6.006 的教学内容里**。讲义 L20（课程总复习）把 Union-Find 明确列为后续课 6.046 的主题。因为它是「实现与数据结构」这条主线绕不开的工具（Kruskal、连通性、等价类），这里作为**补充**给出，标注为课外内容。

接口（不相交集合森林）：

```
make_set(x)      新建只含 x 的集合
find(x)          返回 x 所在集合的代表元
union(x, y)      合并两个集合
```

两个优化：

- **按秩/按大小合并（union by rank/size）**：把小树挂到大树下，树高 ⩽ log n；
- **路径压缩（path compression）**：`find` 时把路径上所有结点直接挂到根。

两者合用，m 次操作的摊还代价是 **O(m·α(n))**，α 是反阿克曼函数，对任何现实输入 α ⩽ 4，可视为 O(1)。

```python
class DSU:
    def __init__(self, n):
        self.p = list(range(n))
        self.r = [0] * n              # 秩
    def find(self, x):
        while self.p[x] != x:
            self.p[x] = self.p[self.p[x]]   # 路径减半（比全压缩更简单）
            x = self.p[x]
        return x
    def union(self, a, b):
        ra, rb = self.find(a), self.find(b)
        if ra == rb: return False
        if self.r[ra] < self.r[rb]: ra, rb = rb, ra
        self.p[rb] = ra
        if self.r[ra] == self.r[rb]: self.r[ra] += 1
        return True
```

- 只做路径压缩：摊还 O(log n)；只做按秩合并：最坏 O(log n)；两者都做：O(α(n))。

- **陷阱**：`find` 写成递归在大数据上会爆栈；`union` 忘了先 `find` 会把两个非根结点互挂，直接破坏结构。

第 21 章 排序专题：一张总表与四条决策规则

21.1 总表

算法	最好	平均	最坏	原地	稳定	额外空间	何时用
插入排序	n	n^2	n^2	是	是	O(1)	n 小、近似有序、在线（边来边排）
选择排序	n^2	n^2	n^2	是	否	O(1)	写操作昂贵、交换次数要少
归并排序	n log n	n log n	n log n	否	是	O(n)	要稳定 + 最坏保证、外部排序、链表排序
堆排序	n log n	n log n	n log n	是	否	O(1)	要原地 + 最坏保证、实时系统
快速排序	n log n	n log n	**n^2**	是	否	O(log n) 栈	实践最快（常数小、缓存友好）
计数排序	n+u	n+u	n+u	否	是	O(n+u)	key 是小整数、u = O(n)
基数排序	n·c	n·c	n·c	否	是	O(n+u)	key 是定长整数/字符串、u 大但位数少
Timsort（Python）	n	n log n	n log n	否	是	O(n)	默认选择，别手写

> 快速排序在 6.006 的讲义里没有单独一讲（L03 只讲 selection/insertion/merge，L05 讲线性排序），但它是 CS170 与 CLRS 第 7 章的核心内容。这里列出来是为了表格完整，细节请读 CS170 笔记第 5 章。

21.2 四条决策规则

1. **要稳定** ⇒ 归并排序 / 计数排序 / 基数排序 / Timsort。堆排序与快排都不稳定。
2. **要原地 + 最坏 O(n log n)** ⇒ 堆排序（代价：缓存不友好、不稳定）。工程上还有 introsort（快排 + 堆排兜底 + 插入排序收尾）。
3. **key 是小范围整数** ⇒ 计数排序；**key 是定长多位数** ⇒ 基数排序（低位先排、每轮稳定）。
4. **近似有序 / n 很小（< 32）** ⇒ 插入排序；实际库都在小块上用插入排序收尾。

21.3 比较模型的下界（一句话）

n! 个输出 ⇒ 决策树高 ⩾ log(n!) = Ω(n log n) ⇒ 任何基于比较的排序最坏情况都不能更快。要突破就必须用「直接寻址」这种分支因子 > 2 的操作（第 5 章）。

21.4 实现检查清单

- [ ] 稳定性：合并时用 `<=`，计数排序用 `append` 到链尾。
- [ ] 递归边界：`b - a > 1` 才继续分，否则死循环。
- [ ] 下标公式：0-index 的 `left/right/parent`。
- [ ] 负数 key：计数排序前先偏移。
- [ ] 大 u：计数排序会爆内存，改基数排序。
- [ ] 比较器：不要给不可比较的混合类型排序。

第 22 章 图专题：表示、遍历与最短路选型

22.1 表示与代价

表示	空间	查边 (u,v)	遍历 u 的出边	适用
邻接表（数组 + 数组）	Θ(V+E)	O(deg(u))	O(deg(u))	默认，顶点点号连续
邻接表（哈希 + 哈希）	Θ(V+E)	**O(1)(e)**	O(deg(u))	任意标号，需要快速查边
邻接矩阵	Θ(V^2)	**O(1)**	O(V)	稠密图、Floyd—Warshall、矩阵运算

线性时间 = Θ(V+E)。所有图算法的复杂度都按这个口径写，不要写成 O(V^2)。

22.2 遍历家族

算法	解决什么	时间	关键不变量
BFS	无权最短路、分层	O(V+E)	按距离递增出队；入队即标记
DFS	可达性、结构信息	O(E)（单源）	完成顺序携带拓扑信息
Full-BFS / Full-DFS	连通分量、全图探索	O(V+E)	对每个未访问点重启
拓扑排序	DAG 的线性化	O(V+E)	逆完成序 = 拓扑序
环检测	判定 DAG	O(V+E)	回边（指向栈上祖先）存在

22.3 最短路选型（本课最重要的一张表）

图的限制	权重	算法	时间	讲义
一般	无权	BFS	O(V+E)	L09
DAG	任意（含负）	DAG Relaxation	O(V+E)	L11
一般	任意	Bellman–Ford	O(V·E)	L12
一般	非负	Dijkstra（二叉堆）	O(E log V)	L13
一般	非负 + 稠密	Dijkstra（数组）	$O(V^2)$	L13
一般	任意	Johnson（APSP）	$O(V^2 \log V + V \cdot E)$	L14
一般	任意	Floyd–Warshall（APSP）	$O(V^3)$	L17

选型口诀：**先问有没有负权，再问是不是 DAG，最后问稠密还是稀疏。**

22.4 三个「坑王」

1. **负权 + Dijkstra**：结果错但不报错。这是最危险的组合。
2. **拓扑序 + 环**：算法给出一个看似合理的顺序，实际无意义。
3. **不可达 + ∞**：`∞ + w` 仍是 ∞（浮点），但用整数哨兵时会溢出；复原路径时也会走进不存在的边。

第 23 章 DP 专题：子问题类型学与复杂度配方

23.1 子问题的五种形状（讲义 L18 结尾的总结表）

子问题形状	例子	状态维度
前缀 / 后缀	Bowling、LCS、LIS、Floyd–Warshall、切杆、子集和	1–2
子串（连续区间）	交替取硬币、算术括号化	2
多序列	LCS	2
整数（值域）	Fibonacci、切杆、子集和	1–2，可能伪多项式
顶点	DAG 最短路、Bellman–Ford、Floyd–Warshall	1–2

23.2 子问题扩张/约束的四种力度（讲义原文分类）

手法	例子	代价
无扩张的约束	LIS（要求包含 A[i]）	子问题数不变
2 倍扩张	取硬币（轮到谁）、括号化（min/max）	×2
Θ(1) 倍扩张	钢琴指法（起始手指）	×F
Θ(n) 倍扩张	Bellman–Ford（边数 k）	×V

23.3 分支因子与复杂度

分支因子	例子	对复杂度的影响
Θ(1)	Fibonacci、Bowling、LCS、取硬币、FW、子集和	总时间 = 子问题数
Θ(degree)	DAG 最短路、Bellman–Ford	出现 $\|E\|$ 因子
Θ(n)	LIS、括号化、切杆	出现 n 因子（常导致 n^3）

配方：总时间 = 子问题数 × 每个子问题的非递归 work。子问题数由状态维度决定，work 由分支因子决定。做题时先把这两个数写出来，再相乘。

23.4 复原方案（parent pointers）

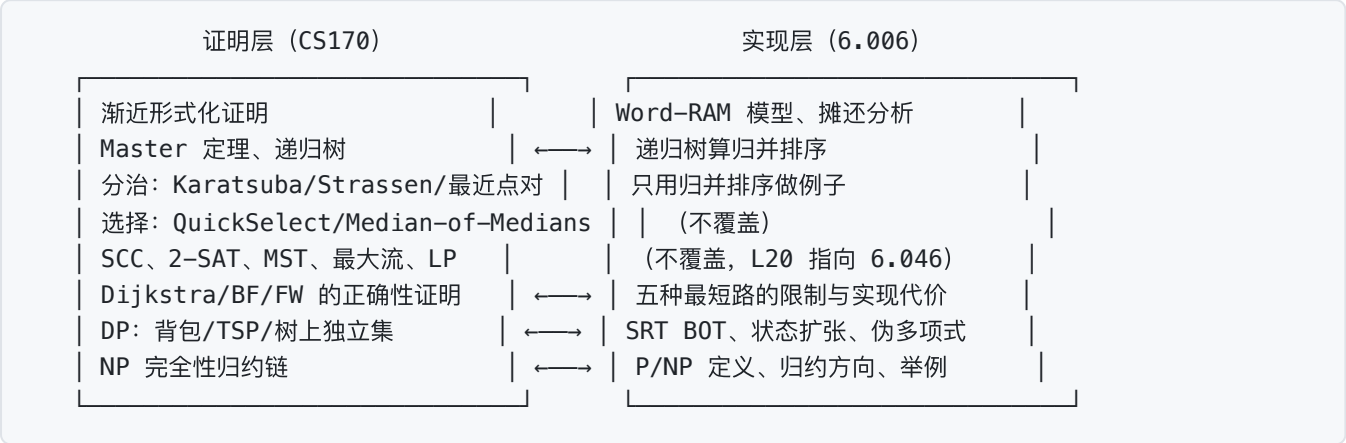
- 一维 DP：`parent[i] = 转移到 i 的那个 j`。
- 二维 DP：`parent[i][j] = (i', j')`，注意 LCS 里「两维都前进」才输出字符。
- 括号化：需要**两个**父指针（左子区间与右子区间），构成一棵二叉树。
- 博弈类：父指针记录「我选了哪一端」。

23.5 DP 的五个常见错误

1. 子问题定义没写清楚（直接写递推）——SRT BOT 的第一步就崩。
2. 拓扑序与递推方向不一致。
3. 基例漏写边界（B(n) 有了，B(n+1) 没有）。
4. 该扩张状态时硬凑（如钢琴指法只存 x(i)）。
5. 把伪多项式当多项式（第 18 章）。

第 24 章 与已有笔记的衔接

24.1 两份笔记的知识互补图



24.2 推荐学习顺序

1. **先读 CS170 第 1 章（渐近）+ 第 2 章（Master 定理）**：6.006 不做形式化推导，直接用结论；没有这块基础，L03 的递归树会看得云里雾里。
2. **再读本笔记第 1–8 章**：这是一条完整的「接口 → 实现 → 增广 → 平衡」主线，CS170 没有对应内容。
3. **图论两边对照读**：本笔记第 9–14 章给实现与选型，CS170 第 6–7 章给 SCC 与正确性证明。**SCC 是 6.006 的空白**，如果目标是面试/竞赛，务必补 CS170 的 Kosaraju。
4. **DP 先读本笔记第 15–18 章**（建模训练更系统：SRT BOT + 子问题扩张），**再读 CS170 第 11–12 章**（背包、TSP、树上 DP 的题型补充）。
5. **最后读 CS170 第 15 章（NP）+ 本笔记第 19 章**：一个给证明，一个给结论与举例。

24.3 三个「只在 6.006 出现」的知识点

- **AVL 的旋转与增广**（第 7 章）：CS170 完全不讲平衡树的实现。
- **线性排序的实现细节**（第 5 章）：计数排序的链式稳定实现、基数排序的位数计算。
- **Johnson 的重赋权**（第 14 章）：CS170 只讲 Dijkstra/BF/FW，不讲 APSP 的势函数技巧。

24.4 两个「只在 CS170 出现」的知识点

- **Master 定理与分治算法族**（Karatsuba、Strassen、最近点对、Median-of-Medians）。

- **MST、最大流最小割、线性规划与对偶**——这些是 6.046 的内容，6.006 明确不覆盖（L20 的 "Next Steps" 页把它们列为后续课程主题）。

第 25 章 复习清单与自测题

25.1 复习清单（按讲次打勾）

数据结构与排序（Quiz 1）：Word-RAM 假设与大整数代价 / 动态数组摊还（r_d < r_i）/ Sequence 与 Set 接口的完整操作表 / 归并排序与 `<=` 的稳定性 / 比较排序下界 Ω(n log n) / 全域哈希族与期望链长 1+(n−1)/m / 哈希表为何不支持 min/max/前驱后继 / 计数排序的稳定实现与基数排序「低位先排」/ `subtree_at` 与 size 增广 / AVL 四种失衡与旋转 / 插入 1 次旋转、删除 O(log n) 次 / 0-index 的 `left/right/parent` / 自底向上建堆 O(n)。

图（Quiz 2）：BFS 入队即标记 / DFS 迭代版 /「逆完成序是拓扑序」/ DAG 松弛 O(V+E) / Bellman-Ford 分层图与 |V|−1 轮 / Dijkstra 懒删除版 O((V+E)log V) / Dijkstra 三种 PQ 实现差异 / Johnson 重赋权与还原公式 / Floyd−Warshall 的 k 在最外层。

DP 与复杂度（Quiz 3 / Final）：SRT BOT 六步 /「加约束」与「加状态」的区别 / LCS、LIS、取硬币、切杆、子集和、括号化、指法的递推 / 多项式 vs 伪多项式 vs 指数 / P、NP、NP-hard、NP-complete 与归约方向 / 5 个 NP 完全问题。

25.2 自测题（16 题，答案在后）

A. 概念判断（对/错，并给一句理由）

1. 动态数组的 `insert_last` 最坏 O(1)。
2. 哈希表的 `find_min` 是 O(1) 期望。
3. AVL 树删除时最多做 1 次旋转。
4. 二叉堆的中序遍历是递增序列。
5. BFS 在带权图上仍能给出最短路。
6. 只要图是 DAG，就可以用 O(V+E) 求最短路，即使有负权边。
7. Dijkstra 在含零权边的图上是正确的。
8. 子集和算法 O(nT) 是多项式时间。
9. 基数排序是稳定的，但计数排序不是。
10. Floyd−Warshall 可以检测负环。

B. 计算与设计

1. n 个元素的动态数组，从空开始做 n 次 `insert_last`，总搬运次数是多少（扩容因子 2）？
2. 用 h(k) = ((7k + 3) mod 11) mod 5 把 {4, 9, 14} 插入长度 5 的链地址哈希表，画出结果。

3. 对图 a→b(2)、a→c(5)、b→c(1)、c→d(2)，从 a 跑 Dijkstra，写出出队顺序与最终距离。
4. 写一个 O(n) 的算法判断有向图是否有环（不要求返回环）。
5. 用 SRT BOT 写出「给一个数组，求最大子数组和」的 DP（Kadane）。
6. 证明：若所有边权为正整数且 ≤ W，则 BFS 的「按层扩张」可以改成按距离扩张，但最坏时间是 O(W·(V+E))，并说明为什么这仍是多项式。

25.3 答案

1. **错**。摊还 O(1)，最坏 O(n)（触发扩容那次）。
2. **错**。哈希表不维护顺序，`find_min` 要扫描全部槽，Θ(n)。
3. **错**。插入最多 1 次；删除可能沿祖先链做 O(log n) 次。
4. **错**。堆只有局部序（父 ≥ 子），中序不是有序序列。
5. **错**。除非所有边权相同（或等价于无权），否则 BFS 给的不是最短路。
6. **对**。DAG 松弛按拓扑序处理，负权不影响正确性，时间 O(V+E)。
7. **对**。非负（含 0）即可；0 权边不会造成「已定稿的点被改进」。
8. **错**。O(nT) 对输入**字数**不是多项式（T 可达 2^w），是伪多项式。
9. **错**。两者都稳定；不稳定的是选择排序、堆排序、快排。
10. **对**。跑完后若存在 d[i][i] < 0，则有负环（可达的）。
11. 扩容发生在 1,2,4,8,…,2^k 处，总搬运 1+2+4+…+n/2 < n，故 **< n 次**（精确地说是 2n−1 量级以内，取决于是否把最后一次算进去；关键是 O(n)）。
12. h(4) = ((28+3) mod 11) mod 5 = (31 mod 11=9) mod 5 = 4；h(9) = ((63+3)=66 mod 11=0) mod 5 = 0；h(14) = ((98+3)=101 mod 11=2) mod 5 = 2。结果：槽 0:[9]、槽 2:[14]、槽 4:[4]。
13. 出队顺序 a(0) → b(2) → c(3)（经 b，2+1=3 < 5）→ d(5)；最终 d[a]=0、d[b]=2、d[c]=3、d[d]=5。
14. 拓扑排序后若处理到的顶点数 < |V| 则有环（Kahn），或 DFS 中出现回边则有环。两者都是 O(V+E)。
15. 子问题 x(i) = 以 A[i] **结尾**的最大子数组和；递推 x(i) = max(A[i], A[i] + x(i−1))；拓扑序 i 递增；基例 x(0) = A[0]；答案 max_i x(i)；时间 Θ(n)。
16. 把每条权 w 的边替换成 w 条单位边（或在 BFS 中用桶按距离分桶），每轮推进 1 个单位距离，最多推进 W·V 轮，每轮 O(V+E)，故 O(W·(V+E))。因为 W 是**输入中出现的整数**，若 W 是常数或多项式于 n，则它是多项式；若 W 可以是指数大（比如 2^n），它就是伪多项式——这正是第 18 章的分界线。

第 26 章 参考来源

26.1 页面（访问日期均为 2026-11-02）

材料	URL
课程主页	https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/
教学大纲（先修、评分、教材版本）	https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/pages/syllabus/
讲义索引（L01–L20 + R01–R19）	https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/pages/lecture-notes/
日历（周次、quiz 周、PS 截止）	https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/pages/calendar/
习题（PS0–PS8：题目 / 模板 / 解答）	https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/pages/assignments/
考试（Quiz 1–3、Final、三份 Review）	https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/pages/exams/

26.2 讲义 PDF

20 份讲义的资源页形如 `.../resources/mit6_006s20_lecN/`（N = 1…20），页内含 `MIT6_006S20_lecN.pdf`；L18 另有一个 `mit6_006s20_lec18.pdf` 的重复入口。全部 20 份均已下载并用 `pdftotext -layout` 转文本后逐讲阅读。标题对应关系见第 0.5 节的讲次地图（L01 Introduction … L19 Complexity、L20 Course Review）。例：

- L01 Introduction：https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/resources/mit6_006s20_lec1/
- L12 Bellman-Ford：https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/resources/mit6_006s20_lec12/
- L18 Pseudopolynomial：https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/resources/mit6_006s20_lec18/

26.3 习题与考试 PDF（题号、分值均取自 PDF 正文）

- **PS0**：先修自测（集合交并差、期望、模运算、两条归纳证明），不含分值标注。
- **PS1**：1-1 渐近函数排序（20 分）、1-2 Sequence 数据结构（16 分）、1-3 Binder Bookmarks（20 分）、1-4 Doubly Linked List（44 分）。
- **PS2**：2-1 解递推（15 分）、2-2 Sorting Sorts（15 分）、2-3 Friend Finder（10 分）、2-4 MixBookTube.tv Chat（15 分）、2-5 Beaver Bookings（45 分）。

- **PS3**：3–1 Hash Practice（5 分）、3–2 Dorm Hashing（15 分）、3–3 The Cold is Not Bothersome Anyway（20 分）、3–4 Pushing Paper（20 分）、3–5 Anagram Archaeology（40 分）。
- **PS4**：4–1 Binary Tree Practice（10 分）、4–3 Gardening Contest（10 分）、4–4 Solar Supply（15 分）、4–5 Robot Wrangling（15 分）、4–6 $\pi z^2 a$ Optimization（40 分）。（4–2 未出现在抓取文本的题目标题行，故未列入。）
- **PS5**：5–1 Graph Practice（15 分）、5–2 Power Plants（10 分）、5–3 Short–Circuitry（10 分）、5–4 Ancient Avenue（10 分）、5–5 Statum Quest（15 分）、5–6 6.006 Tilt（40 分）。
- **PS6**：6–1 Dijkstra Practice（15 分）、6–2 Short Circuits（10 分）、6–3 Dynamite Detonation（20 分）、6–4 Conservative Cycles（10 分）、6–5 SparkPlug Derby（20 分）、6–6 Johnson's Algorithm（25 分）。
- **PS7**：7–1 Effective Campaigning（15 分）、7–2 Caged Cats（15 分）、7–3 Odd Paths（15 分）、7–4 Pizza Partitioning（15 分）、7–5 Shorting Stocks（40 分）。
- **PS8**：8–1 Oil Well that Ends Well（25 分）、8–2 Splits Bowling（25 分）、8–3 Quarter Partition（25 分）、8–4 Corrupt Chronicles（25 分），并要求标注每题属多项式 / 伪多项式 / 指数。
- **Quiz 1**（120 分 / 120 分钟）、**Quiz 2**（90 分 / 90 分钟）、**Quiz 3**（60 分 / 60 分钟）、**Final**（180 分 / 180 分钟）的题目构成见第 0.4 节表格；每次考试另配官方解答 PDF。

26.4 抓取说明

- **成功**：上述 6 个页面全部抓取成功；L01—L20 共 20 份讲义 PDF、PS0—PS8 共 9 份题目 PDF、Quiz 1/2/3 与 Final 共 4 份 PDF 全部下载并转文本。
- **未抓取**：R01—R19 复习课讲义、`/pages/practice-problems/`、`/pages/quizzes/` 正文。凡本笔记提到「复习课会做」的内容（如 `build` 可达 O(n)），均按讲义正文的转述写出，未逐条核对复习课 PDF。
- **教材**：OCW 大纲列 CLRS 第三版（ISBN 9780262033848）；本笔记的章节号按第四版给出，图论章节号同时标注两版（20/22、21/23、22/24、23/25 章）。
- **并查集**：不属于 6.006 教学内容（L20 将其列为 6.046 主题），第 20.6 节作为补充给出，来源为通用教材知识。